Computer Vision 1

Neighborhood Processing
Image Filtering
Fourier Transform

Lecturer: Yen-Chia Hsu

Sep 2026

Credit: This slide is inherited from Dr. Martin Oswald and Dr. Dimitris Tzionas

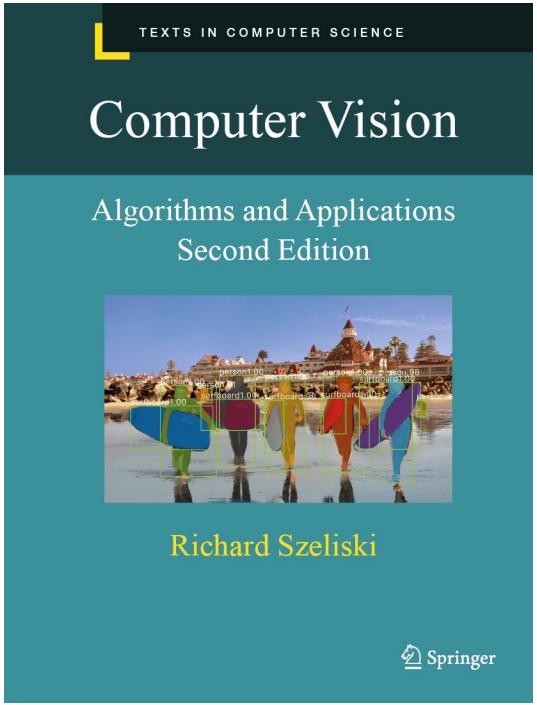
University of Amsterdam

Outline

- **Neighborhood image processing**
  - Connected Component Analysis
  - Mathematical Morphology (Dilation, Erosion, Opening, Closing)
- **Linear Image Filtering**
  - Cross-Correlation and Convolution
  - Frequently used Filters
  - Gaussian Filter
- **Nonlinear Image Filtering**
  - Median Filter
- **Frequency Domain Analysis**
  - Fourier Transform

Textbook

- 3.2
- 3.3
- 3.4 (updated on syllabus)



[https://szeliski.org/Book/]

Outline

- **Neighborhood image processing**
  - Connected Component Analysis
  - Mathematical Morphology (Dilation, Erosion, Opening, Closing)
- **Linear Image Filtering**
  - Cross-Correlation and Convolution
  - Frequently used Filters
  - Gaussian Filter
- **Nonlinear Image Filtering**
  - Median Filter
- **Frequency Domain Analysis**
  - Fourier Transform

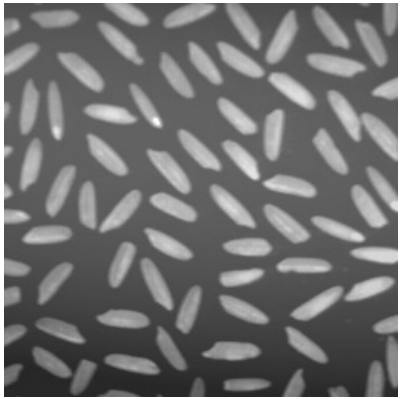
Simple Image Processing Examples

Instance Segmentation and Counting

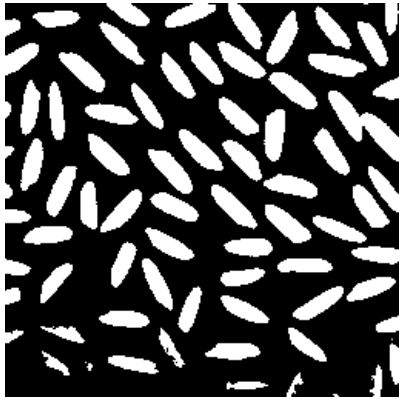
- Rice grains

- Organs

Input Image



Binarization

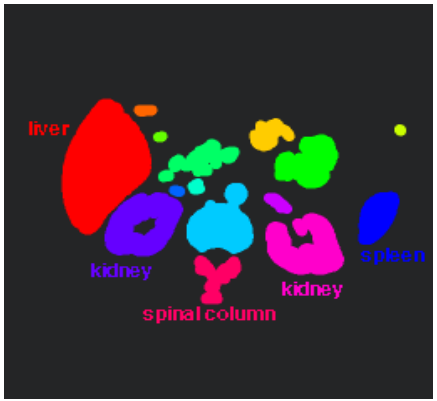


Instance Labeling









Thresholding (for Binarization)

Binarize input signal:





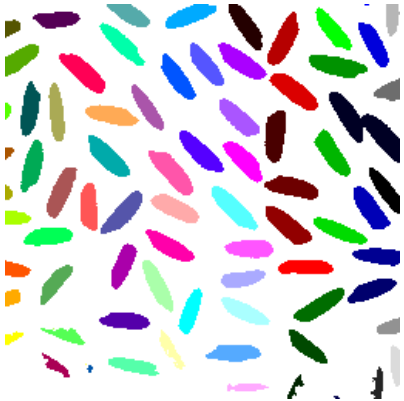
Input Image

Binarized Image $B(i,j) = \begin{cases} 255 & \text{if } I(i,j) > \tau \\ 0 & \text{otherwise} \end{cases}$ Threshold

Connected Components Analysis

- Connected Components Analysis is a multiple-class labeling operation.
- It performs the change from pixels to regions.
- All pixels in the same region are given the same label.



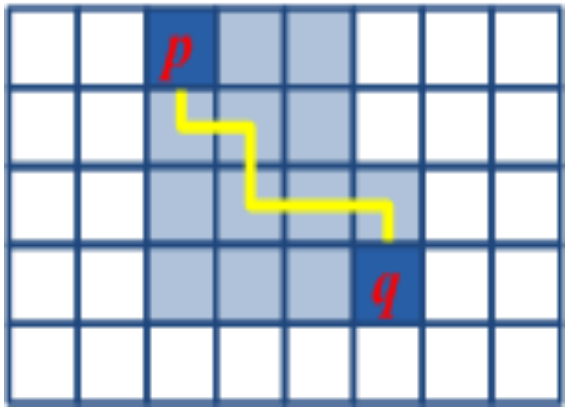


Connected Component Analysis

Definition of Connected Component

– Two pixels p and q belong to the same connected component C if there is a sequence of 1-pixels $(p_0, p_1, \ldots p_n)$, where

- $p_0 = p$
- $p_n = q$
- $p_{i-1}, p_i : i = 1, \ldots, n$ are neighbors

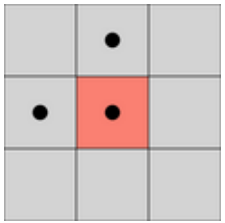


Region Property Computation

For each connected region, we can compute many of its properties, e.g.

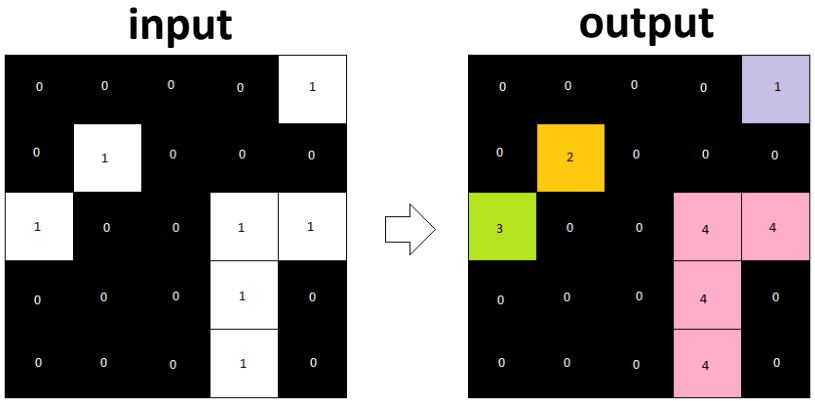
- areas
- perimeter
- centre of gravity
- circularity,
- major axis
- minor axis
- mean and standard deviation of radial distance
- bounding box
- extremal axis length from bounding box
- second order moments (row, column, mixed)
- …

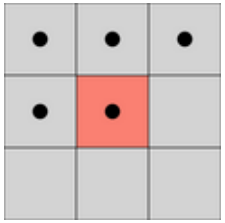
Connected Component Analysis



4-connectivity
(no diagonal)

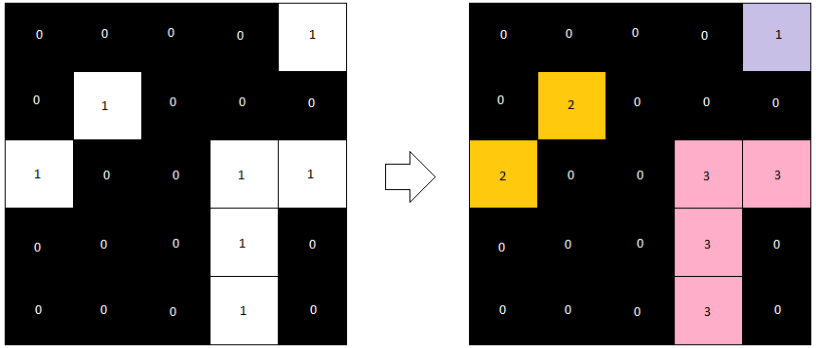
4 connectivity





8-connectivity
(all pixels nearby)

8 connectivity



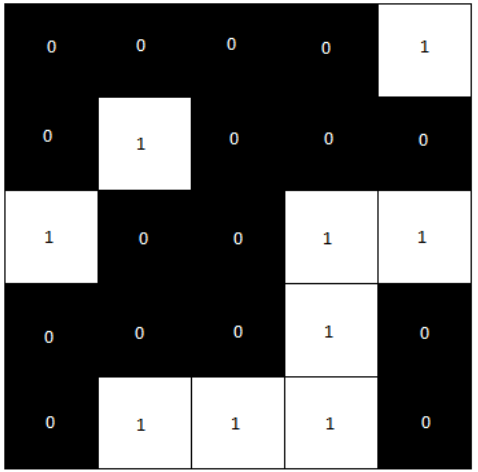
Source: https://medium.com/data-science/implementing-a-connected-component-labeling-algorithm-from-scratch-94e1636554f

Connected Component Analysis

First Pass (8-connectivity):

1. Iterate through each element of the data by column and by row
2. If the element is not the background
    1. Get the neighboring elements of the current element
    2. If there are no **neighbors**, uniquely label the current element and continue
    3. Otherwise, find the neighbor with the smallest label and assign it to the current element
    4. Store the equivalence between neighboring labels

Set ID	Equivalent Labels
1	1
2	2
3	3,4
4	3,4



Equivalency List

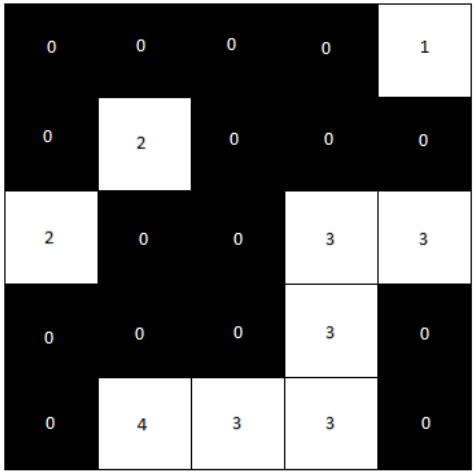
Source: https://medium.com/data-science/implementing-a-connected-component-labeling-algorithm-from-scratch-94e1636554f

Connected Component Analysis

Second Pass (8-connectivity):

1. Iterate through each element of the data by column and by row
2. If the element is not the background
    1. Relabel the element with the lowest equivalent label

Set ID	Equivalent Labels
1	1
2	2
3	3,4
4	3,4



Equivalency List

4 -> 3

Source: https://medium.com/data-science/implementing-a-connected-component-labeling-algorithm-from-scratch-94e1636554f

Mathematical Morphology

In binary image processing, mathematical morphology consists of two basic operations,

dilation , erosion

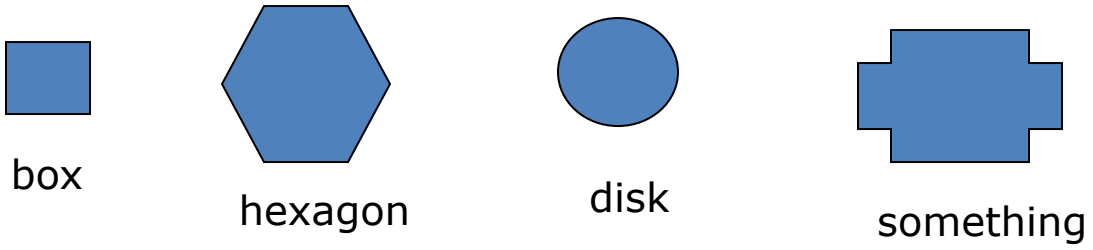
and several composite operations

closing , opening, . . .

Structuring Elements (i.e., kernel)

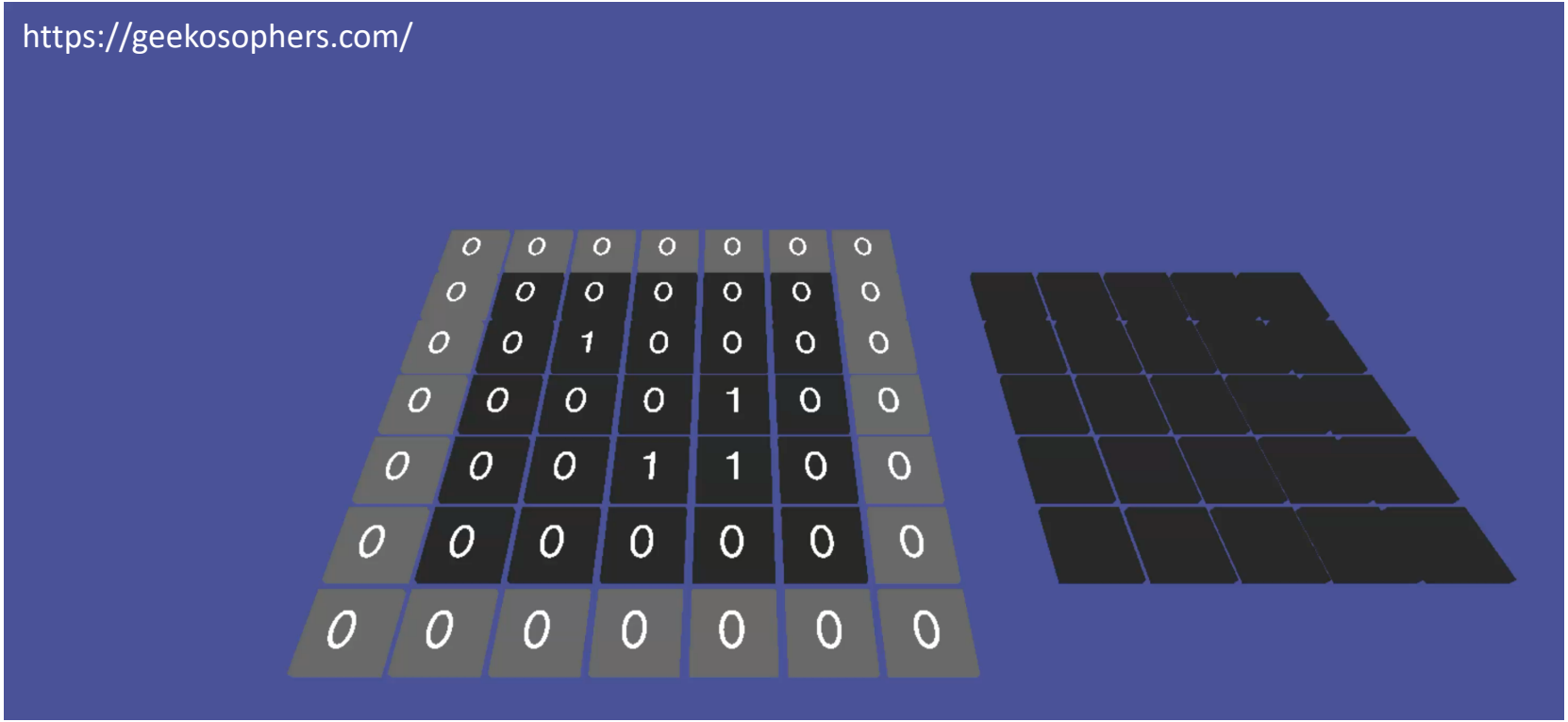
For morphological operations, we need a structuring element (i.e., kernel), which is a shape mask.

They can be any shape and size that is digitally representable, and each has an origin.



Dilation

Dilation takes binary image B, places the origin of structuring element K over each pixel, and takes the maximum value in B over the pixel locations defined by the mask (using 1s).

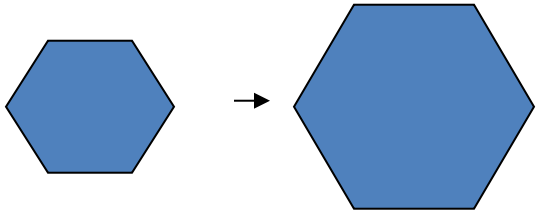


Dilation

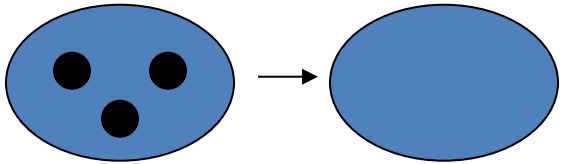
Dilation expands the connected sets of 1s of a binary image.

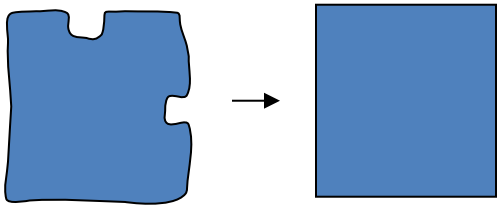
It can be used for

1. growing features



2. filling holes and gaps





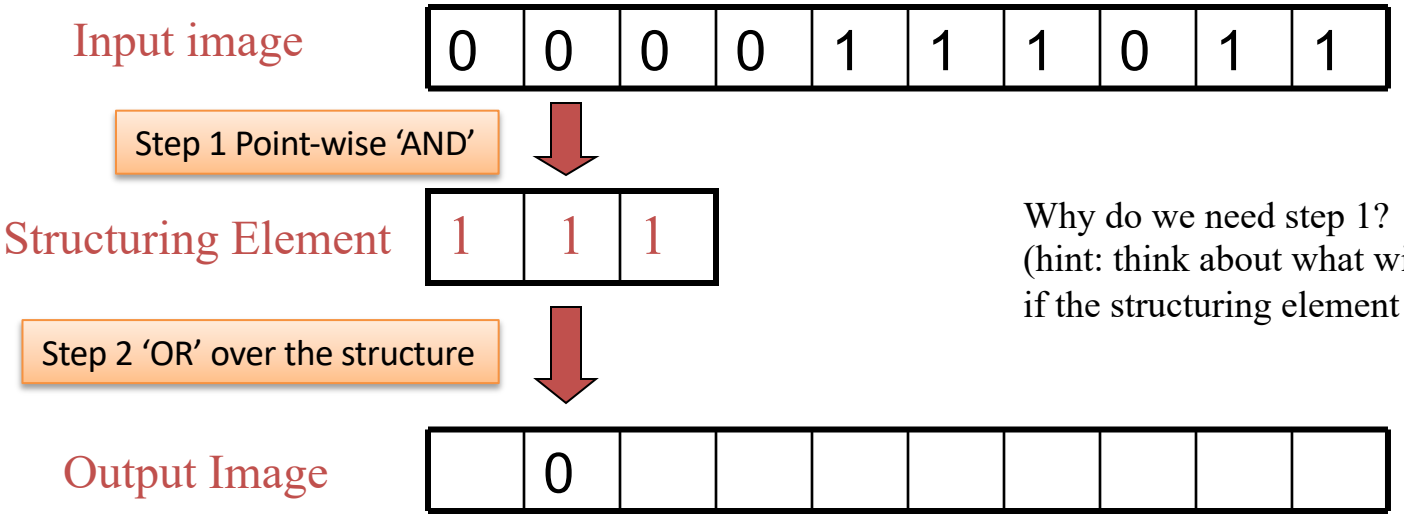
Example for Dilation

AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



Why do we need step 1?
(hint: think about what will happen if the structuring element has 0s)

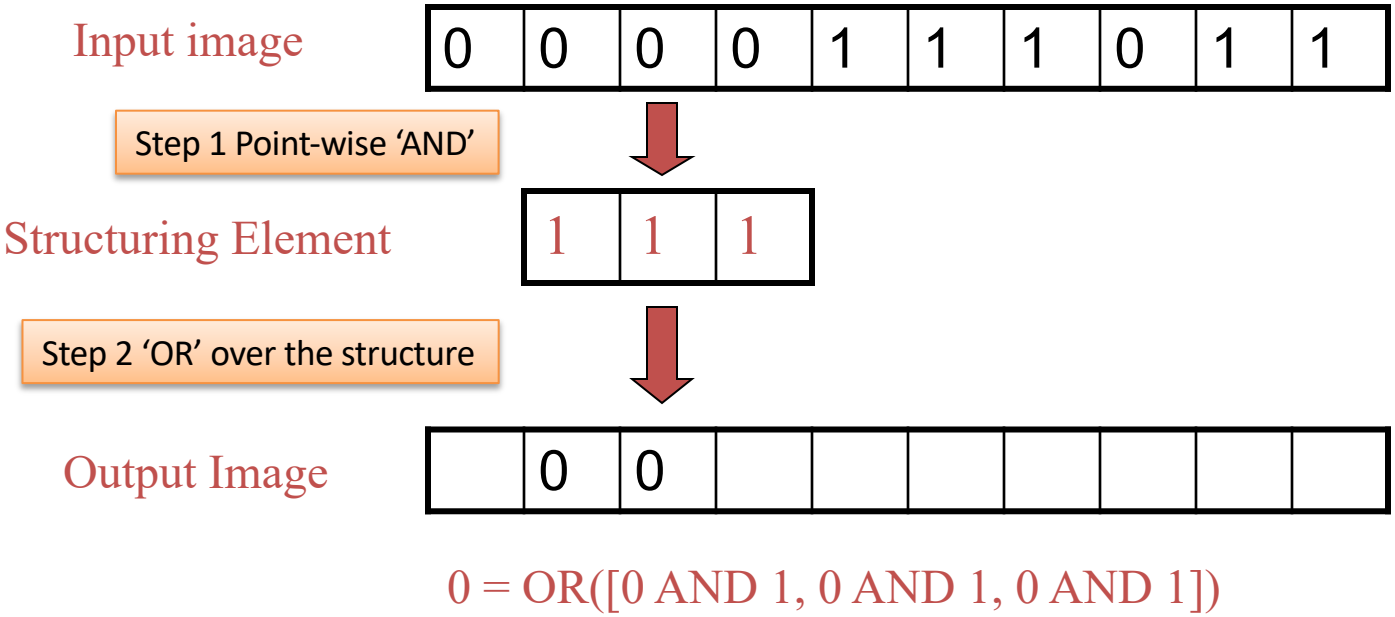
Example for Dilation

AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



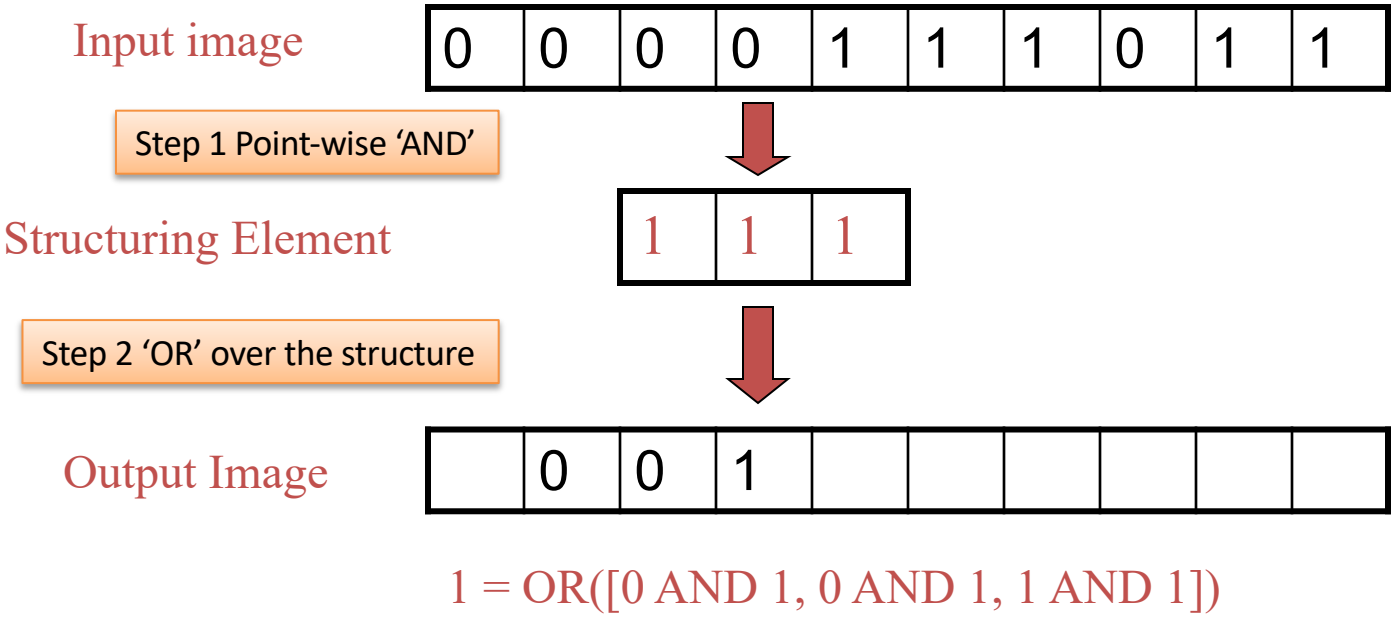
Example for Dilation

AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



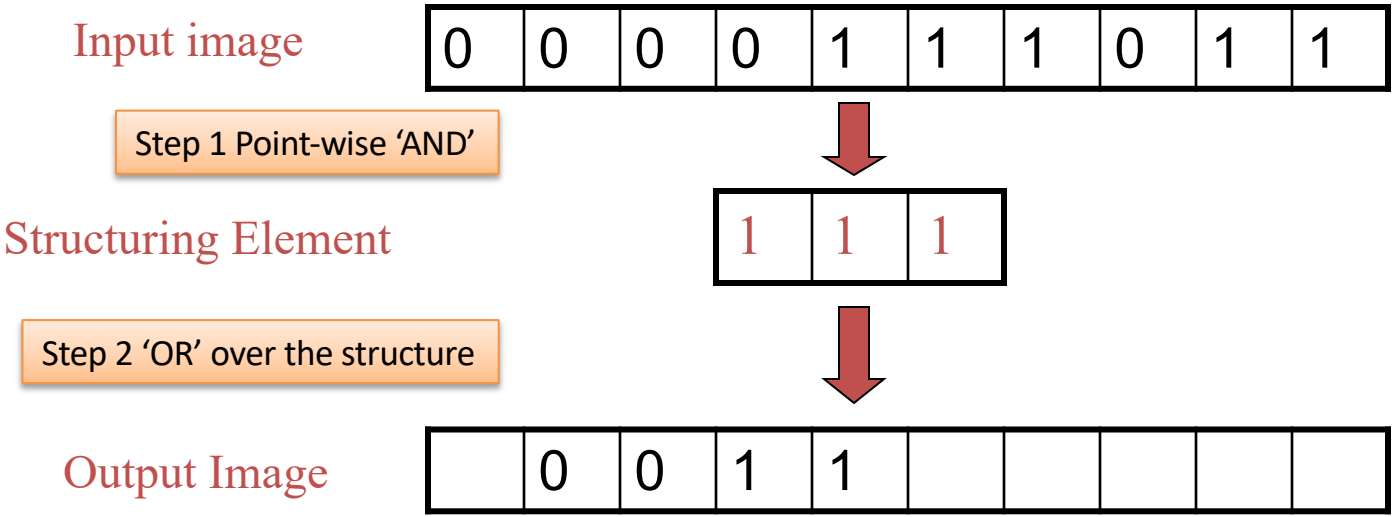
Example for Dilation

AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



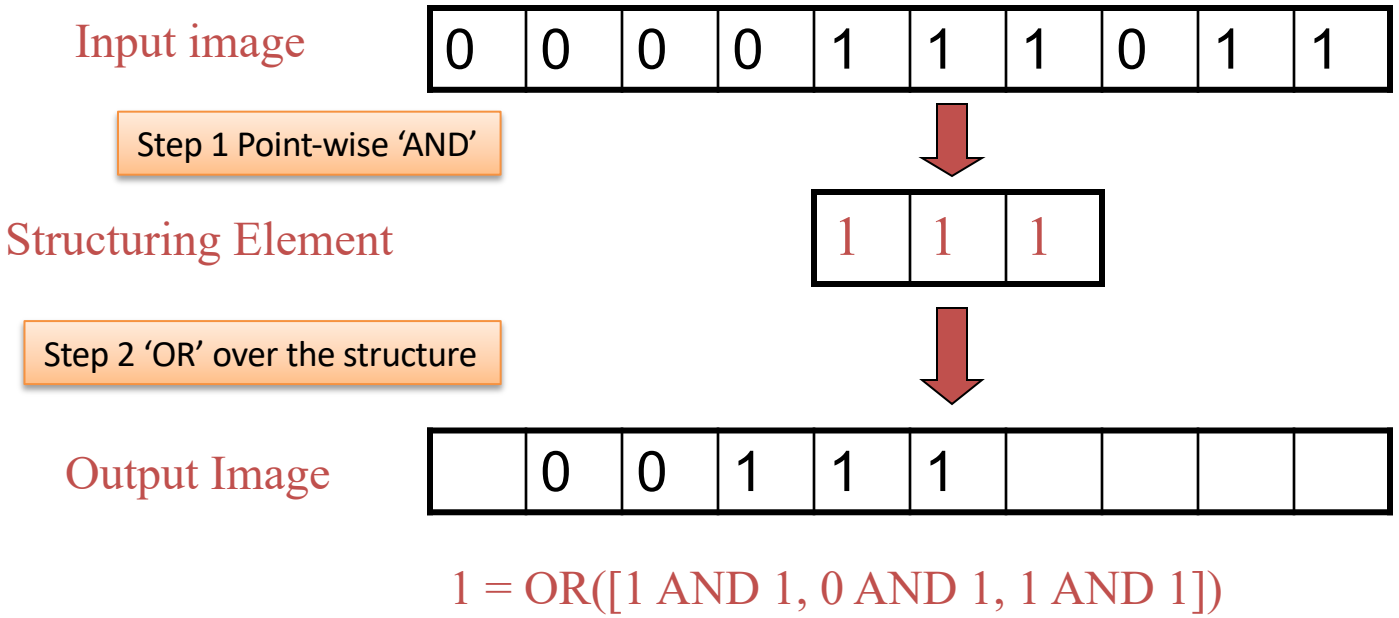
Example for Dilation

AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



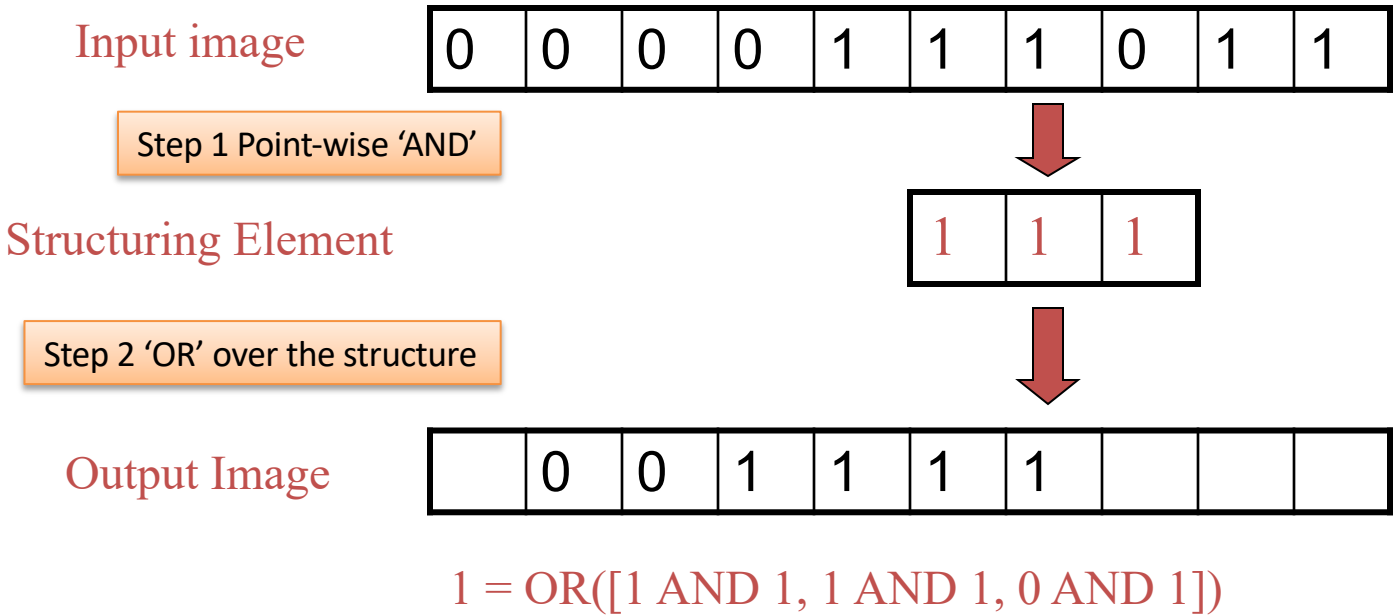
Example for Dilation

AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



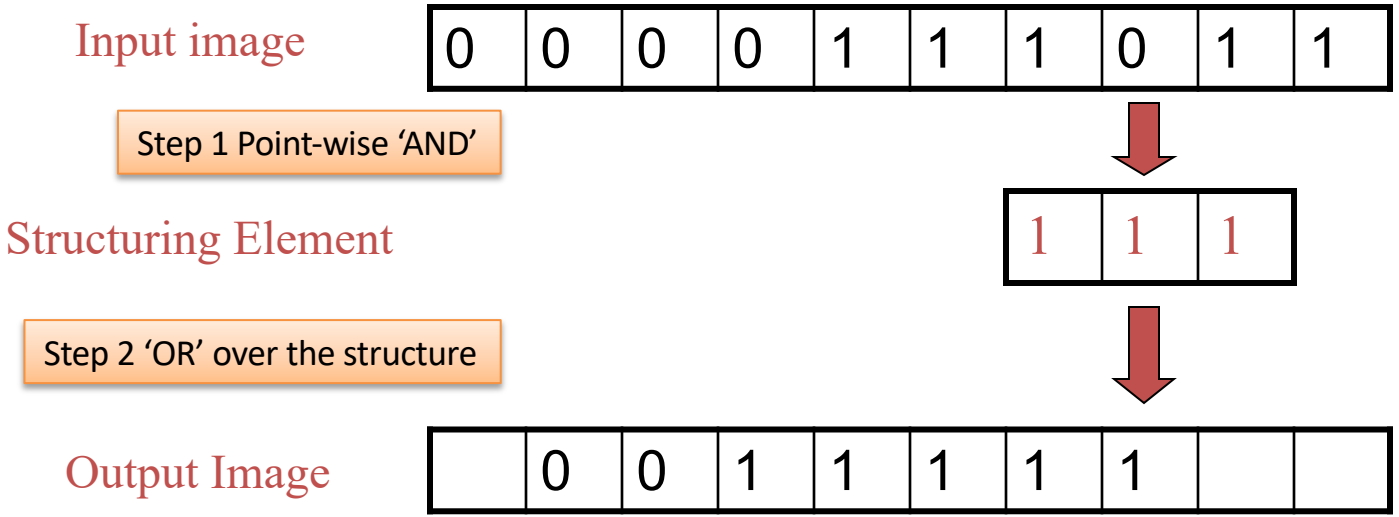
Example for Dilation

AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



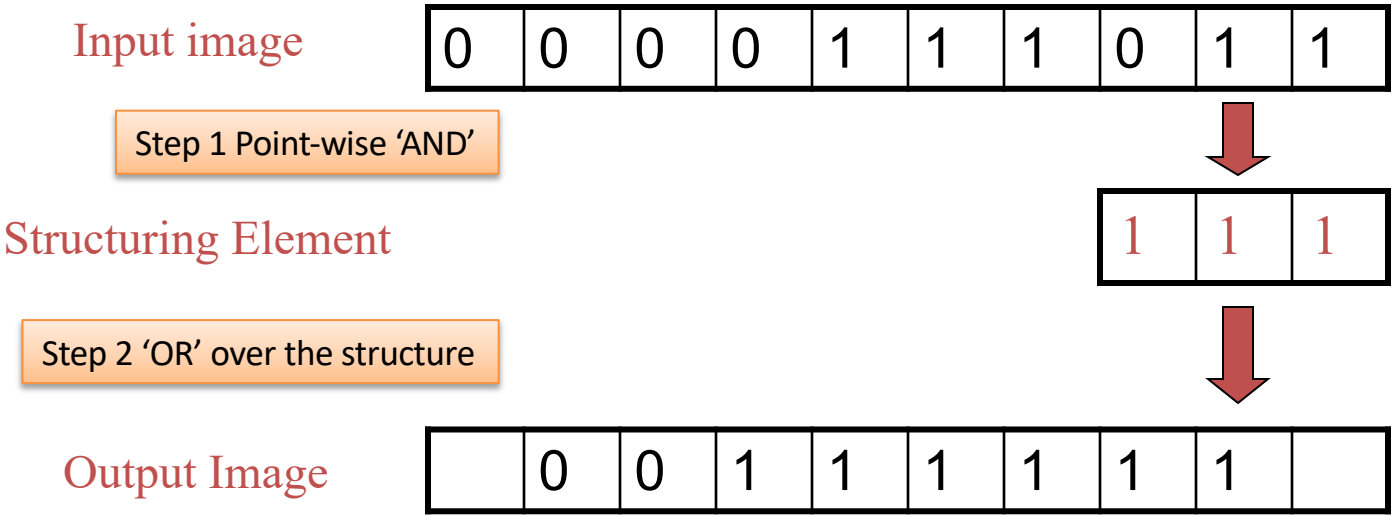
Example for Dilation

AND

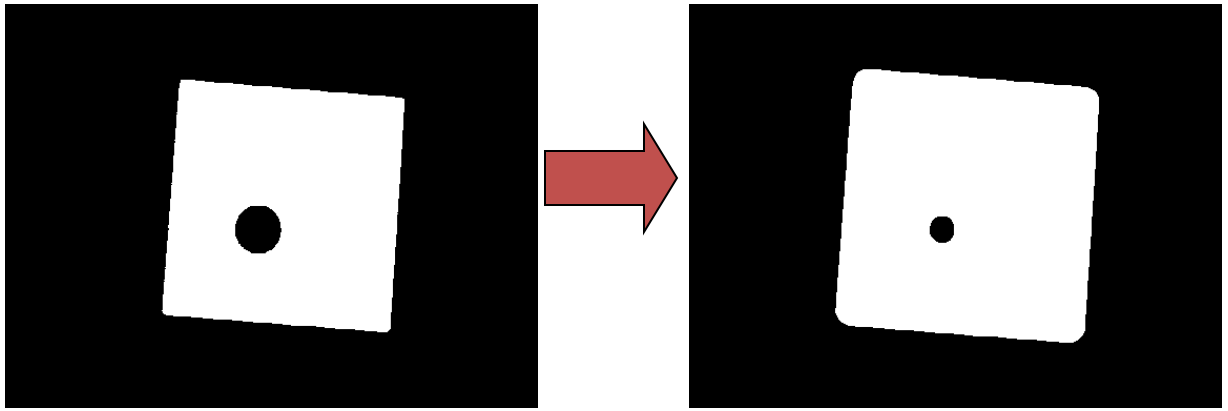
x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



Another Dilation Example

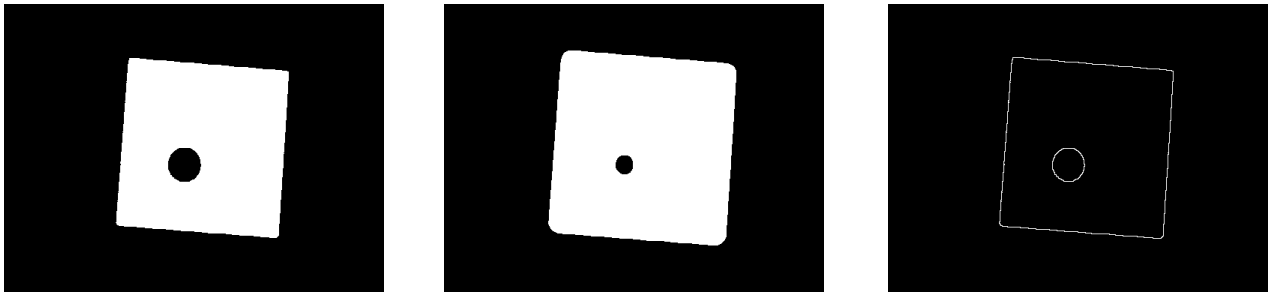


- Image get lighter, more uniform intensity

Edge detection example

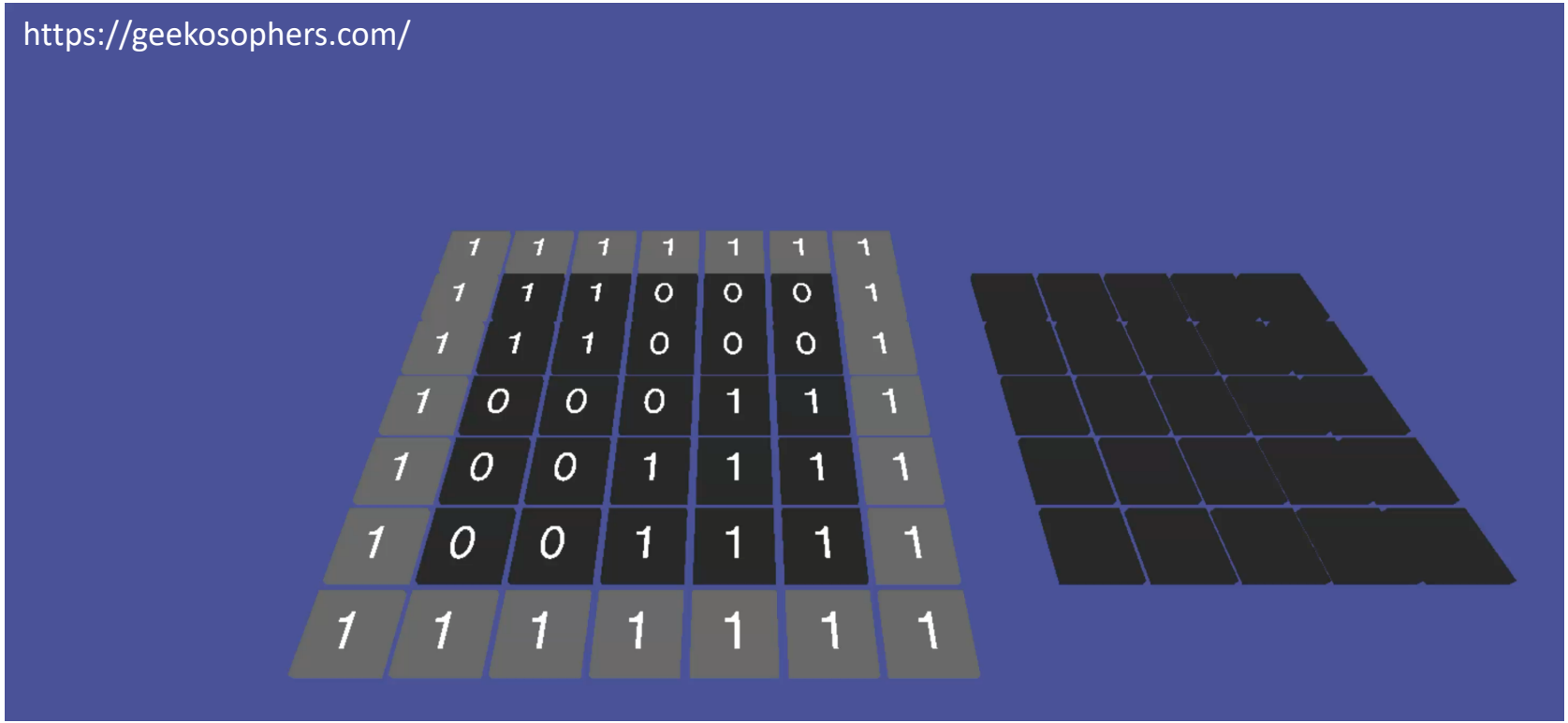
Edge Detection

1. Dilate input image
2. Subtract input image from dilated image
3. Edges remain!



Erosion

Erosion takes a binary image B, places the origin of structuring element K over each pixel, and takes the minimum value in B over the pixel locations defined by the mask (using 1s).

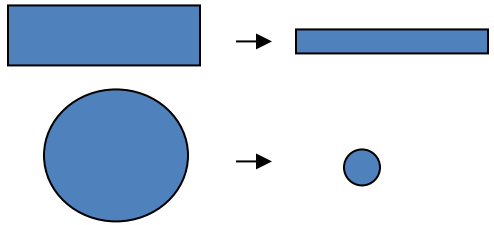


Erosion

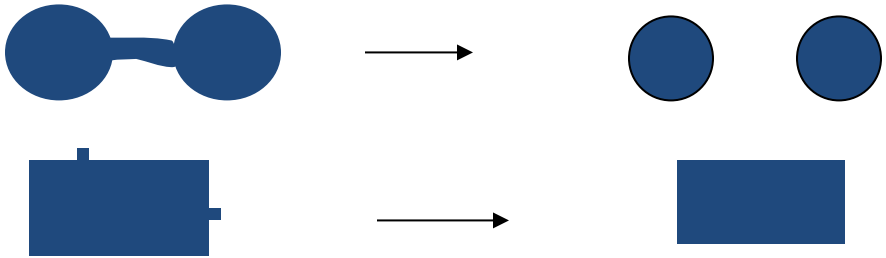
Erosion shrinks the connected sets of 1s of a binary image.

It can be used for

1. shrinking features



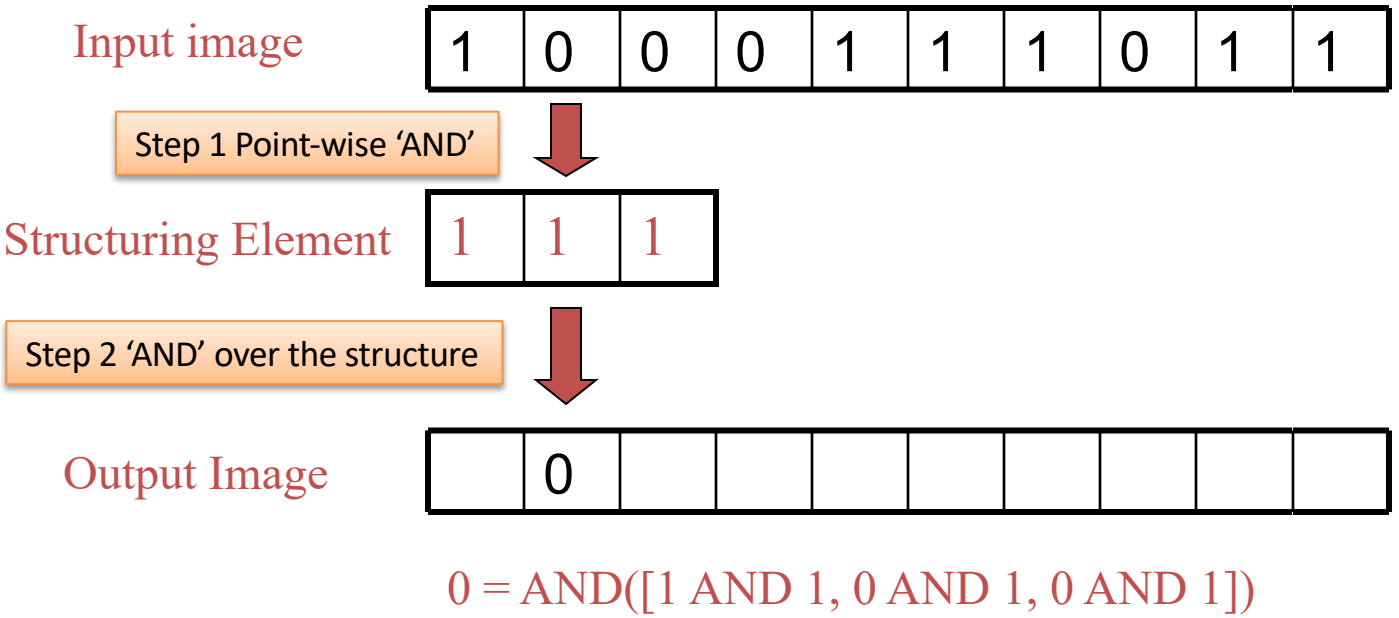
2. Removing bridges, branches and small protrusions



Example for Erosion

AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

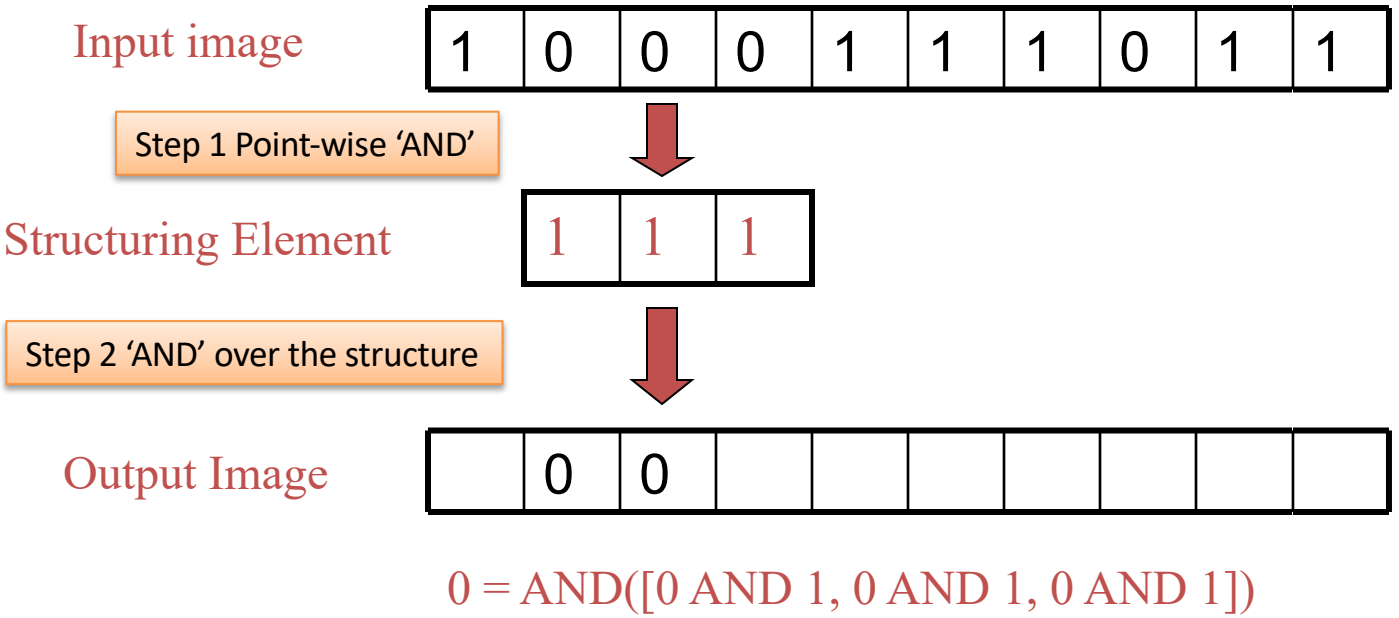


0 = AND([1 AND 1, 0 AND 1, 0 AND 1])

Example for Erosion

AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

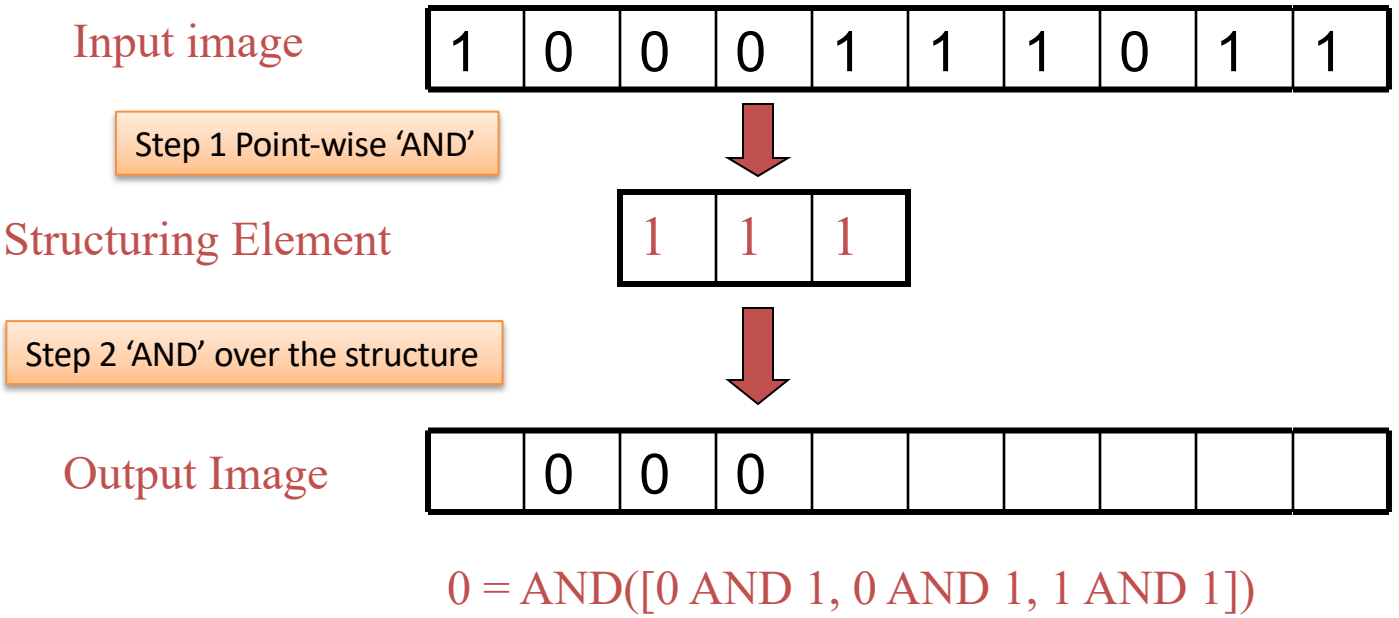


0 = AND([0 AND 1, 0 AND 1, 0 AND 1])

Example for Erosion

AND

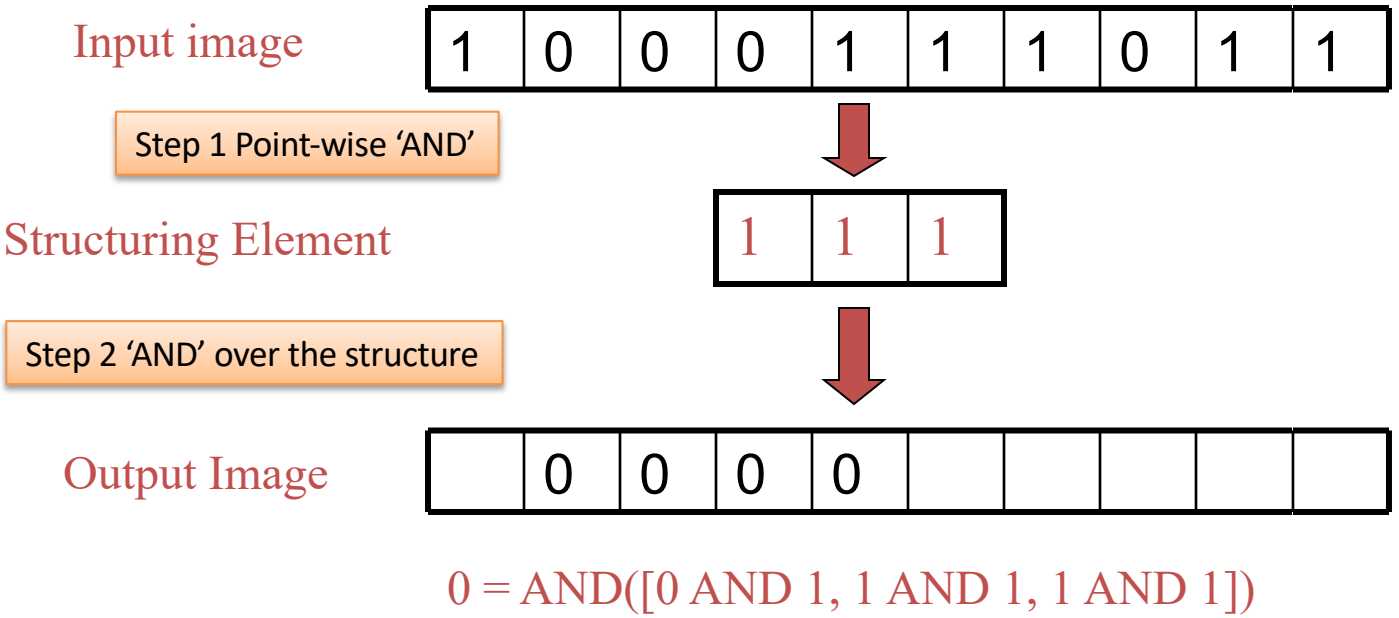
x	y	F
0	0	0
0	1	0
1	0	0
1	1	1



Example for Erosion

AND

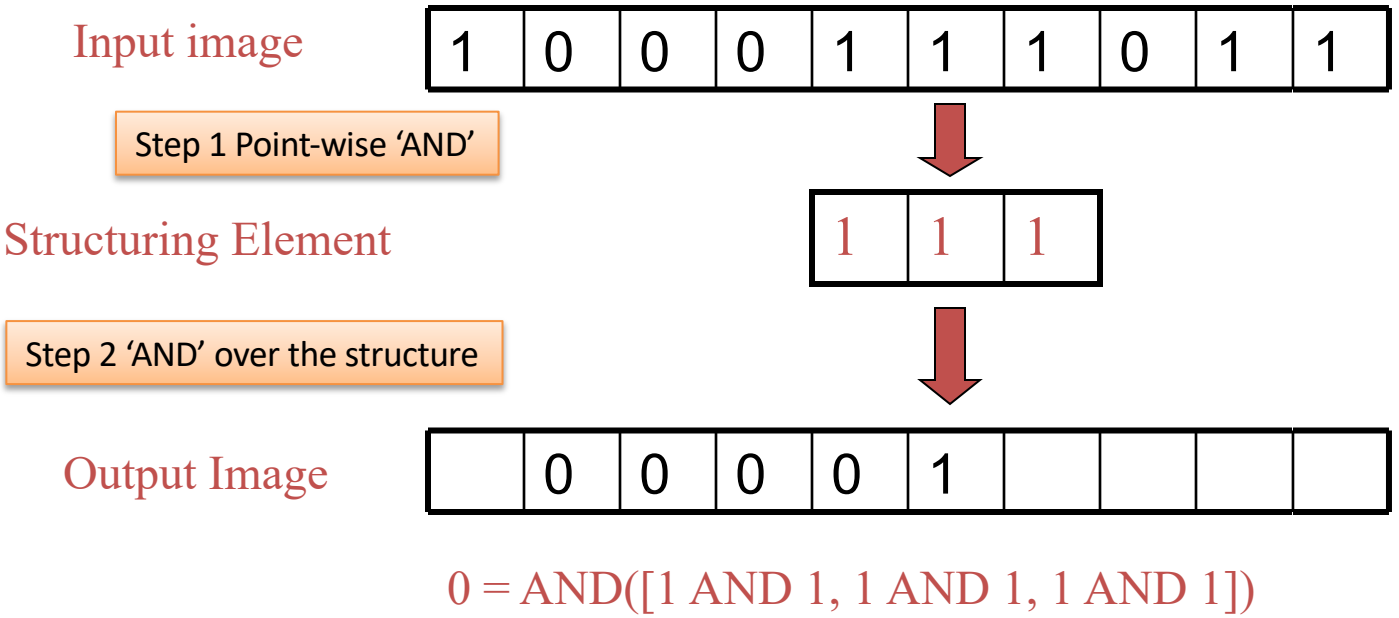
x	y	F
0	0	0
0	1	0
1	0	0
1	1	1



Example for Erosion

AND

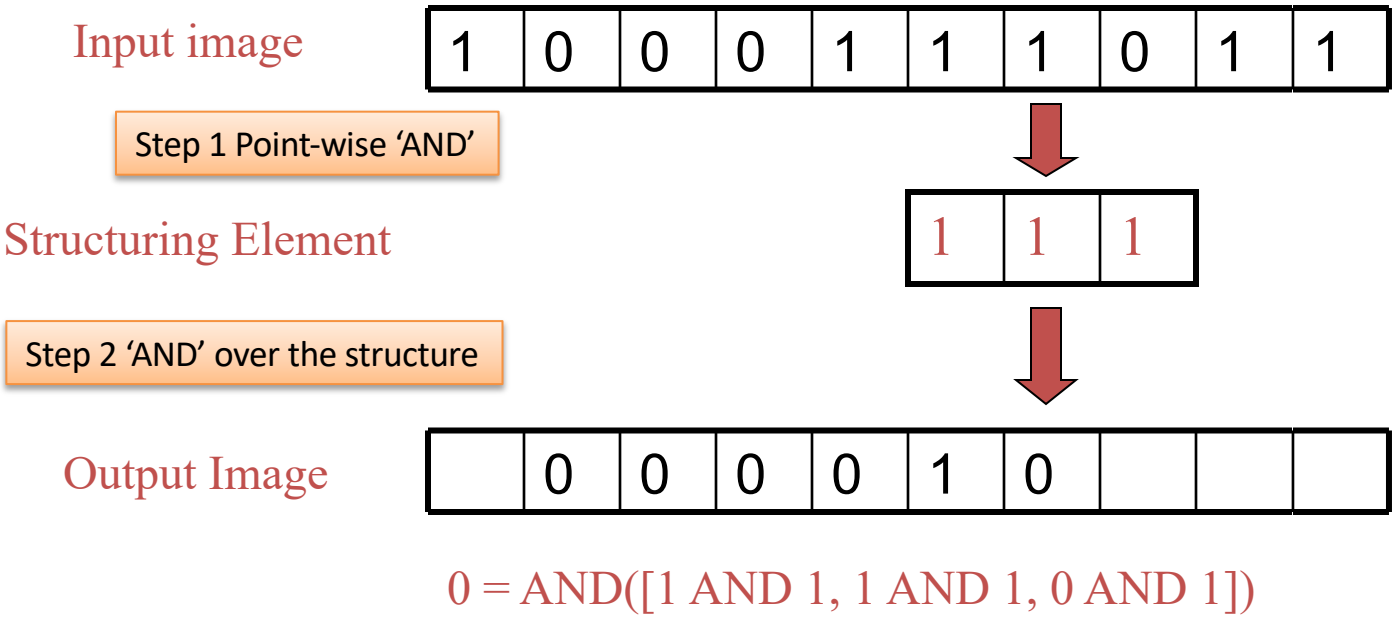
x	y	F
0	0	0
0	1	0
1	0	0
1	1	1



Example for Erosion

AND

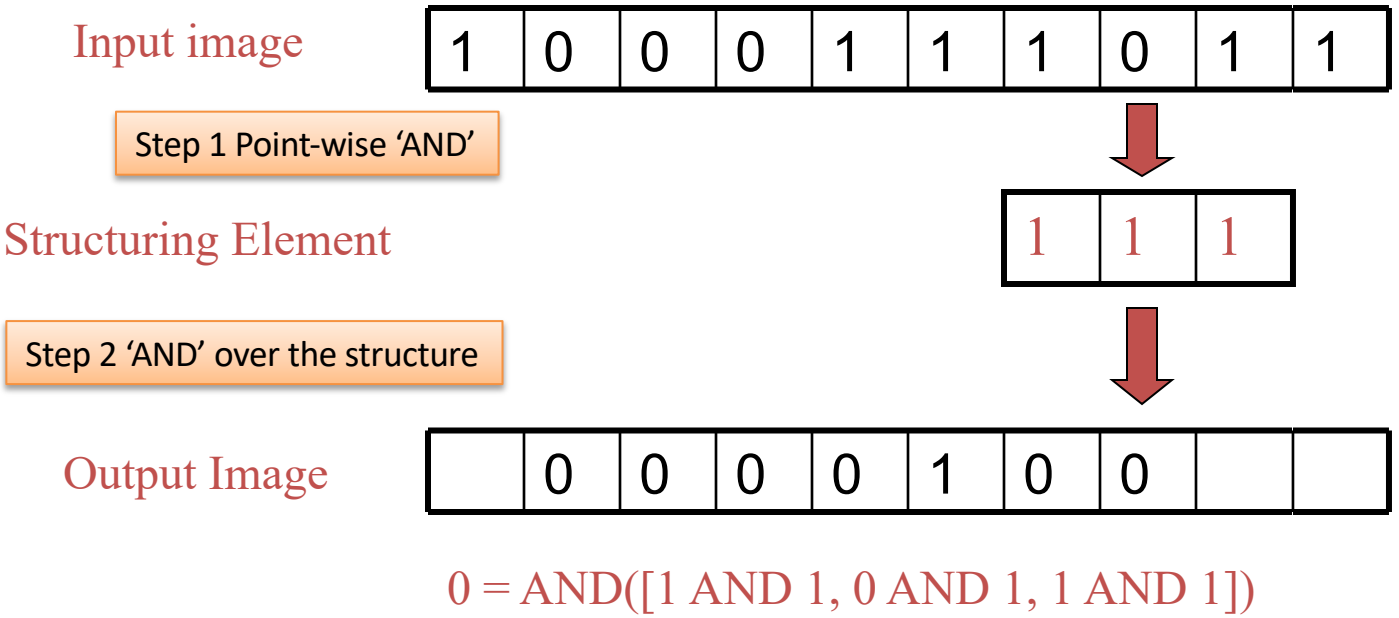
x	y	F
0	0	0
0	1	0
1	0	0
1	1	1



Example for Erosion

AND

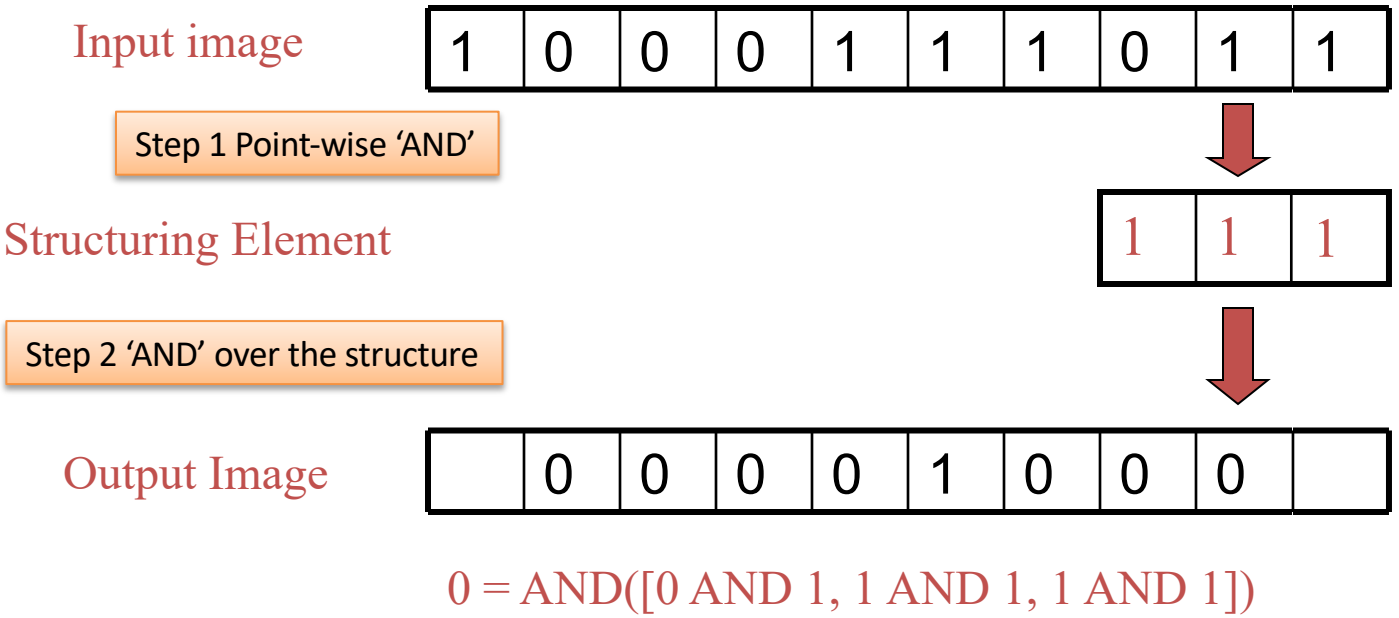
x	y	F
0	0	0
0	1	0
1	0	0
1	1	1



Example for Erosion

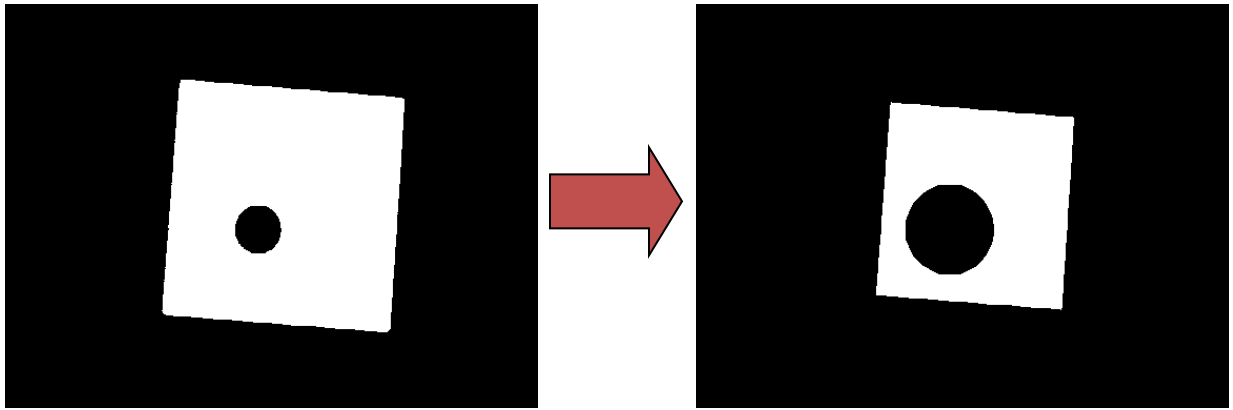
AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1



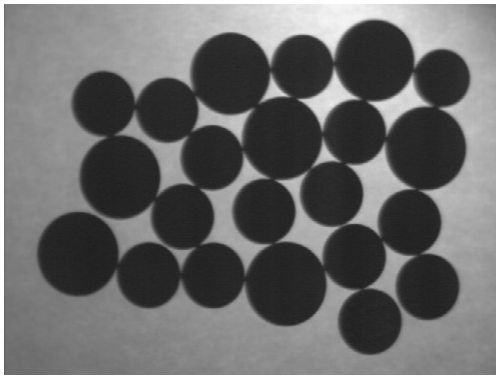
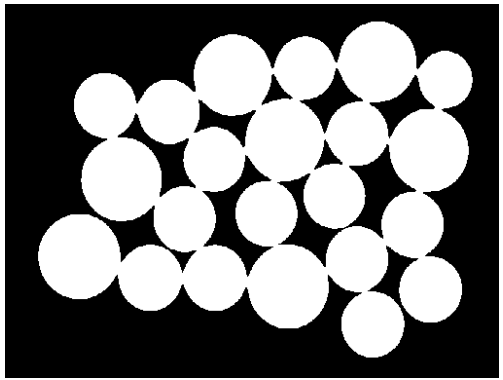
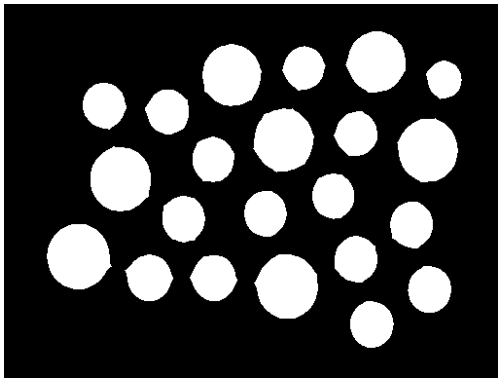
An example of erosion

- White = 1, black = 0, applying image erosion gets another darker image



Counting Coins

- Counting coins is difficult if they touch each other!
- Solution: Binarization and Erosion separates them! And then you can apply the connected component algorithm to count them.

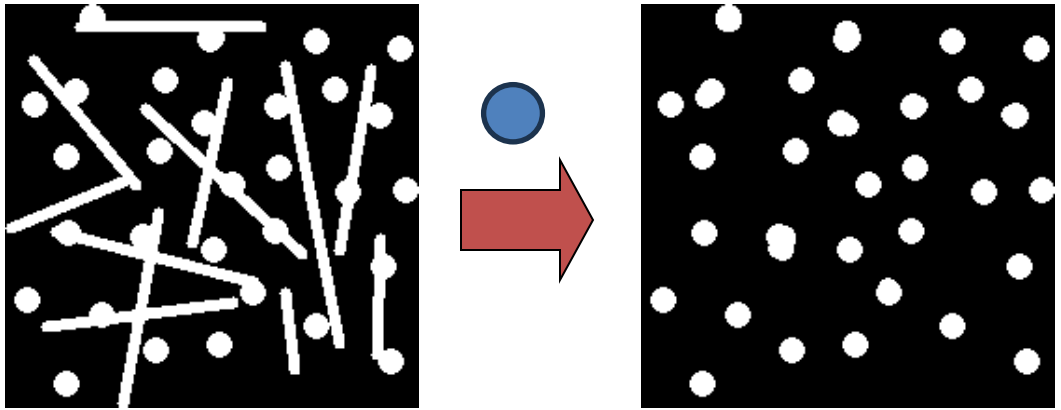
Opening

- Similar to Erosion
  - Spot and noise removal
  - Less destructive
- Erosion first and then dilation

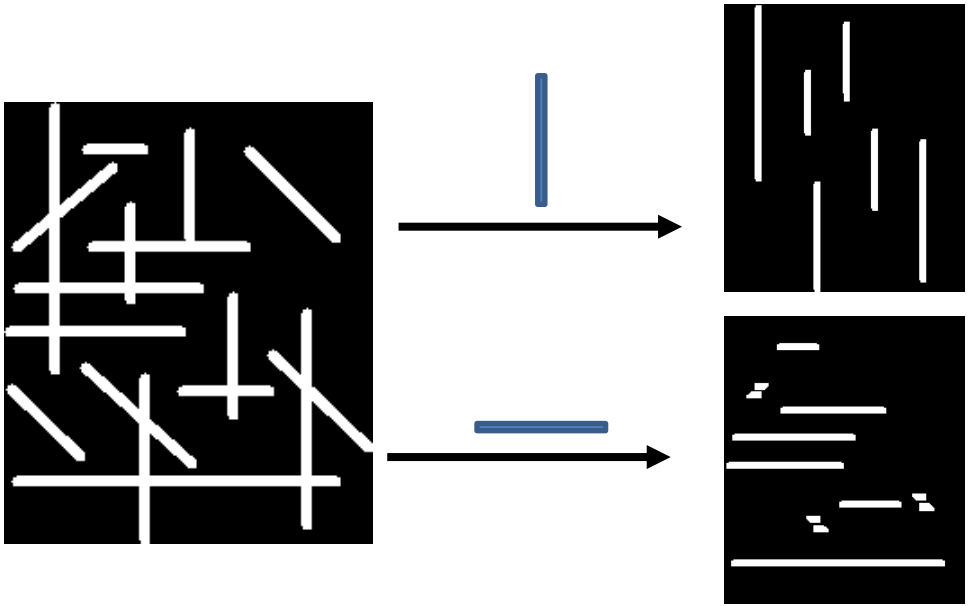
- We use the s*ame structuring element for both operations.*

Opening Example

- Opening with an 11-pixel diameter disc
- Erosion first and then dilation

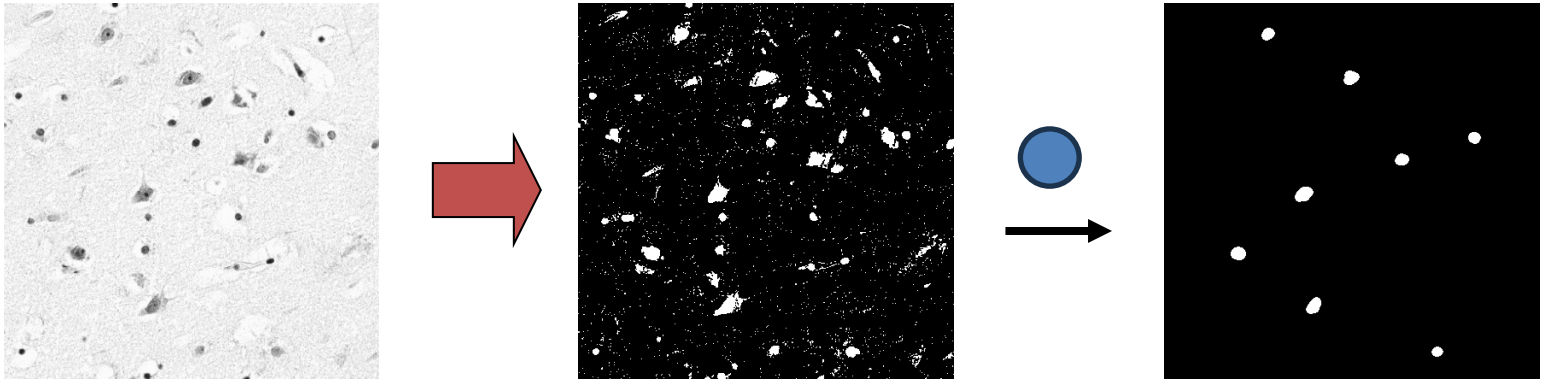


Opening Example



Use Opening for Separating Blobs

- Use large structuring element that fits into the big blobs
- Structuring Element: disc



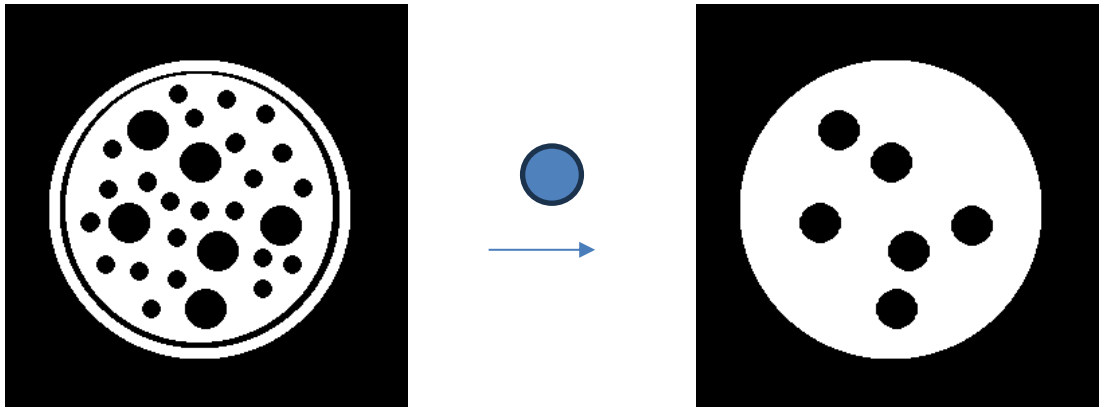
Closing

- Similar to Dilation
  - Removal of holes
  - Tends to enlarge regions, shrink background
- Dilation first and then erosion.

- We use the s*ame structuring element for both operations.*

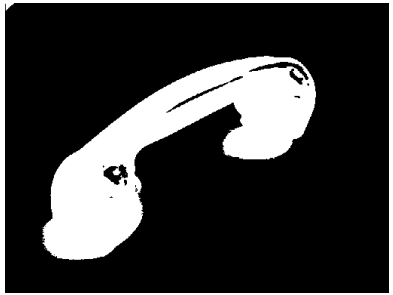
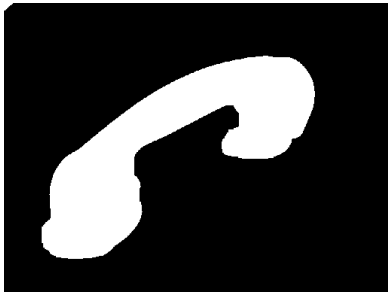
Closing Example

- Closing operation with a 22-pixel disc
- Closes small holes in the foreground



Closing Example

1. Threshold
2. Closing with disc of size 20

Outline

- Neighborhood image processing
  - Connected Component Analysis
  - Mathematical Morphology (Dilation, Erosion, Opening, Closing)
- **Linear Image Filtering**
  - Cross-Correlation and Convolution
  - Frequently used Filters
  - Gaussian Filter
- Nonlinear Image Filtering
  - Median Filter
- Frequency Domain Analysis
  - Fourier Transform

Linear Filtering

A linear filter can be written as $$g(i,j) = \sum_{k,l} f(i+k, j+l) \cdot h(k,l)$$



$f(x,y)$ → H → $g(x,y)$

H is a function that linearly transforms image f to g

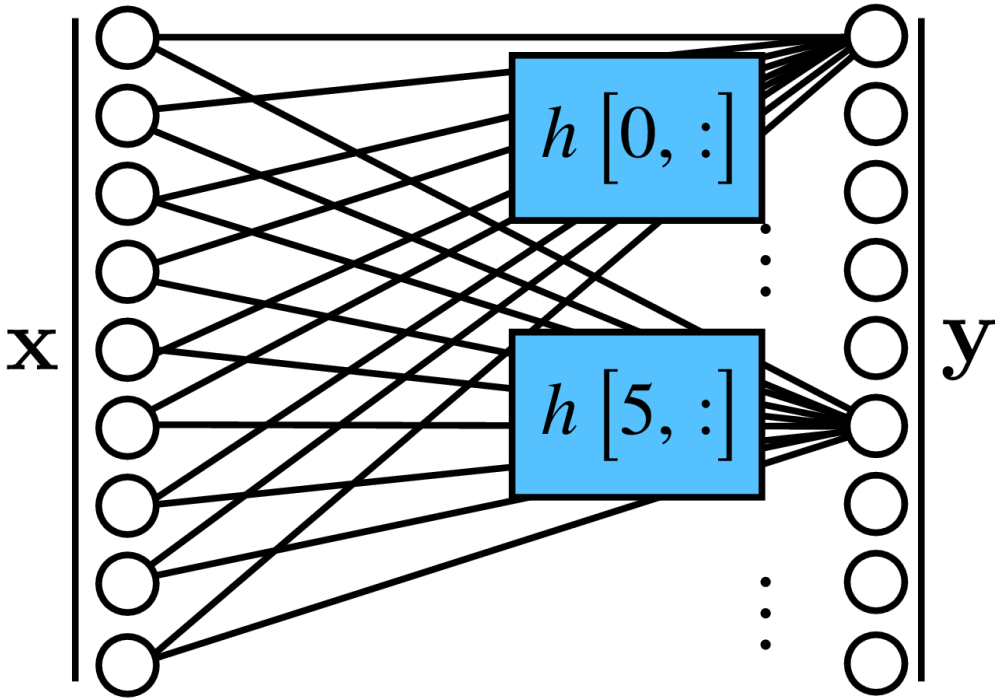


Linear, if and only if: $$\mathrm{H}\big(a(i,j) + b(i,j)\big) = \mathrm{H}\big(a(i,j)\big) + \mathrm{H}\big(b(i,j)\big)$$

$$\mathrm{H}\big(C \cdot a(i,j)\big) = \mathrm{C} \cdot \mathrm{H}\big(a(i,j)\big)$$

Linear Filtering

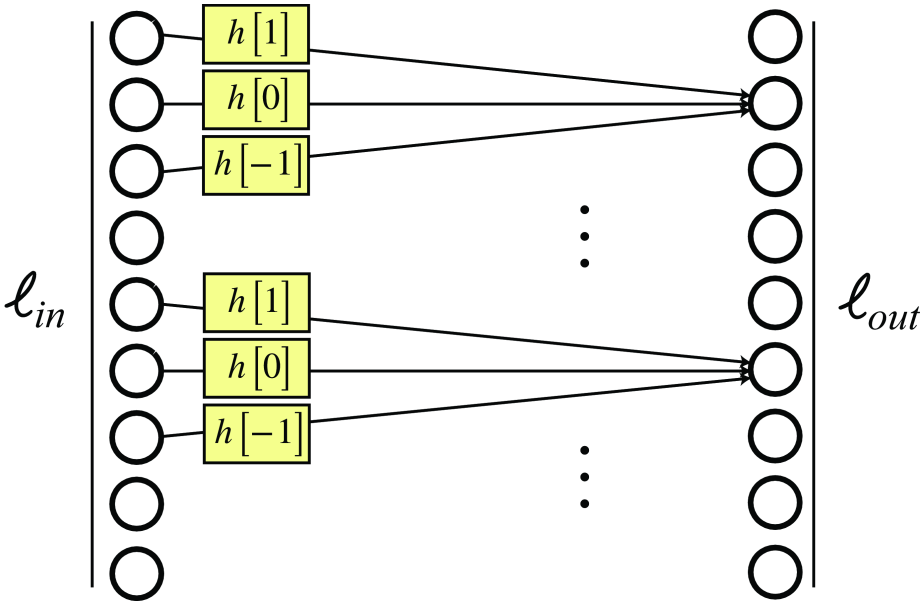
H is a function with weights h that linearly transforms input x to output y



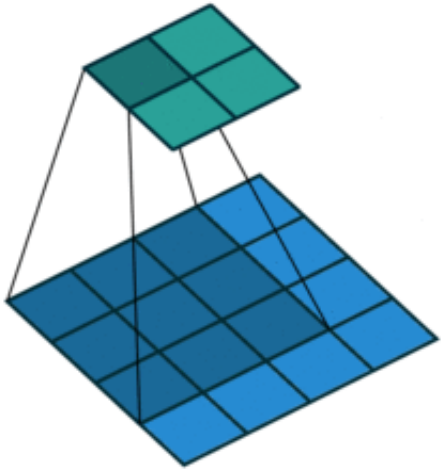
Source: https://visionbook.mit.edu/linear_image_filtering.html

Convolution

The convolutional operation is a special case of linear filter that looks at only a local neighborhood.

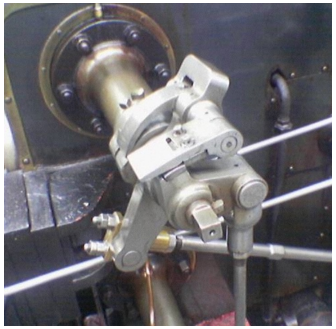


Source: https://visionbook.mit.edu/linear_image_filtering.html



Source: https://github.com/vdumoulin/conv_arithmetic

Convolution example



Input image

Discrete convolution: sum of the element-wise multiplication

Input

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

Filter (Kernel)

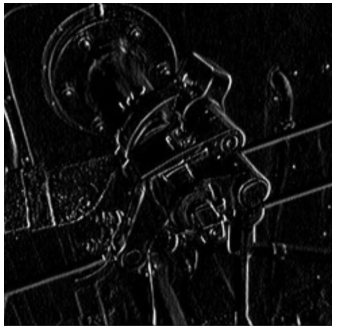
1	0	-1
1	0	-1
1	0	-1

=

Output

6		

7x1+4x1+3x1+
2x0+5x0+3x0+
3x-1+3x-1+2x-1
= 6

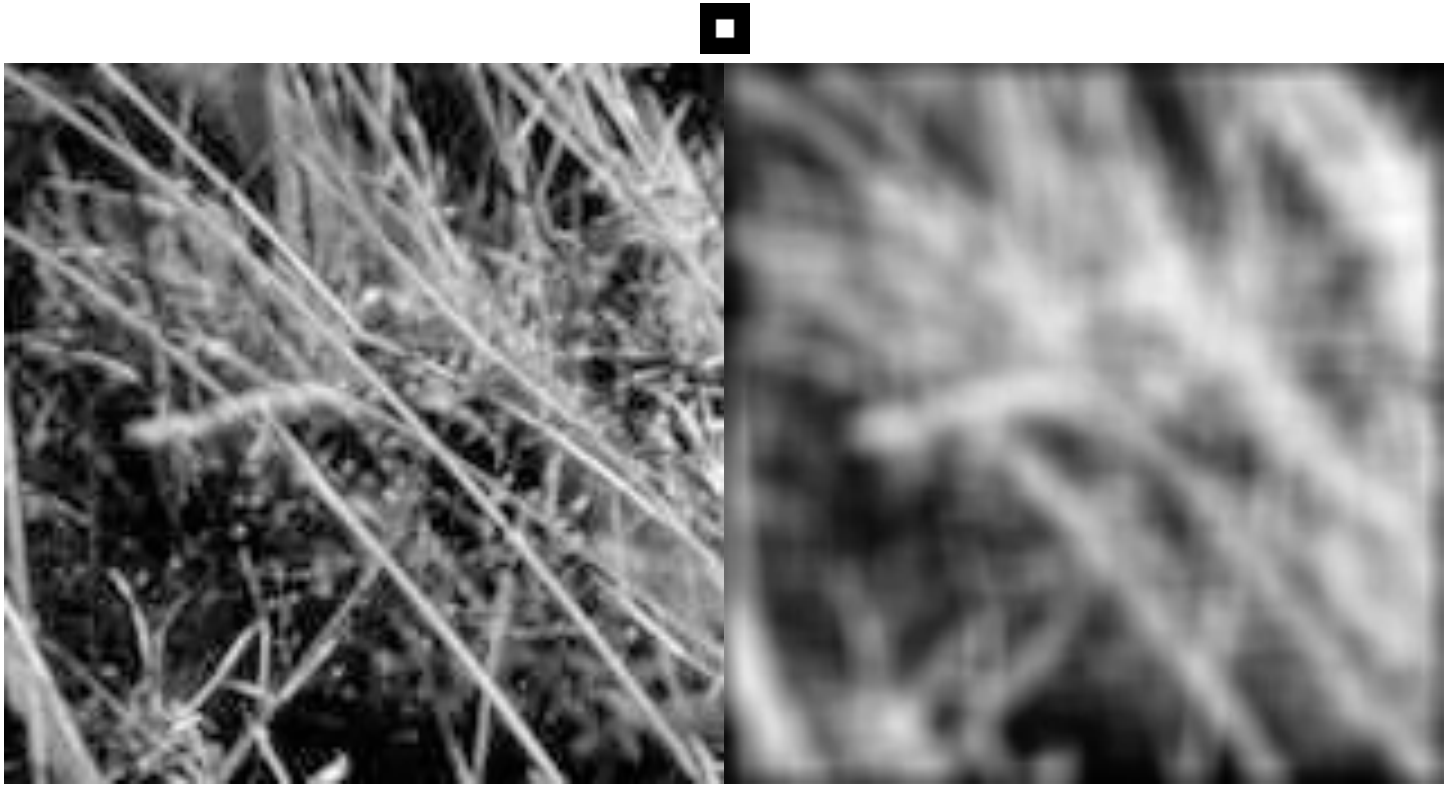


Output image

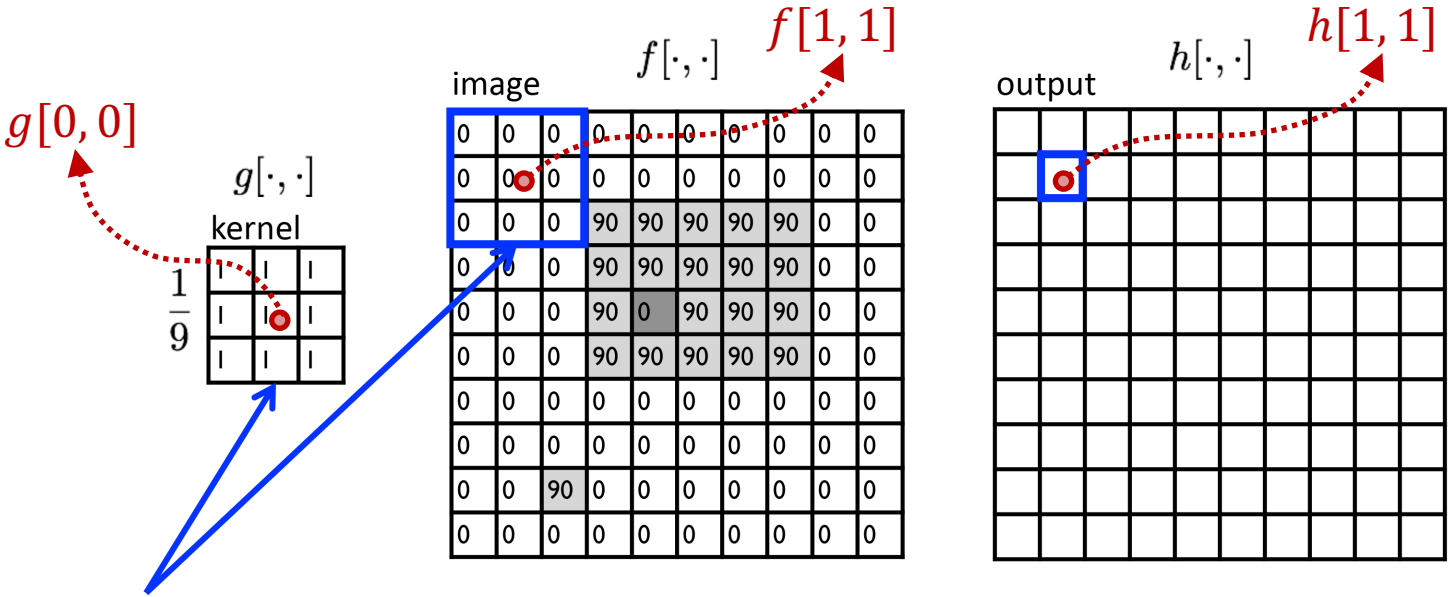
Note: mathematically speaking, this slide is doing cross-correlation and not convolution (for convenience, modern libraries do cross-correlation but call these convolution filters)

Source -- https://docs.opencv.org/4.x/dc/ddf/tutorial_how_to_use_OpenCV_parallel_for_new.html

Box filter



Let's run the box filter



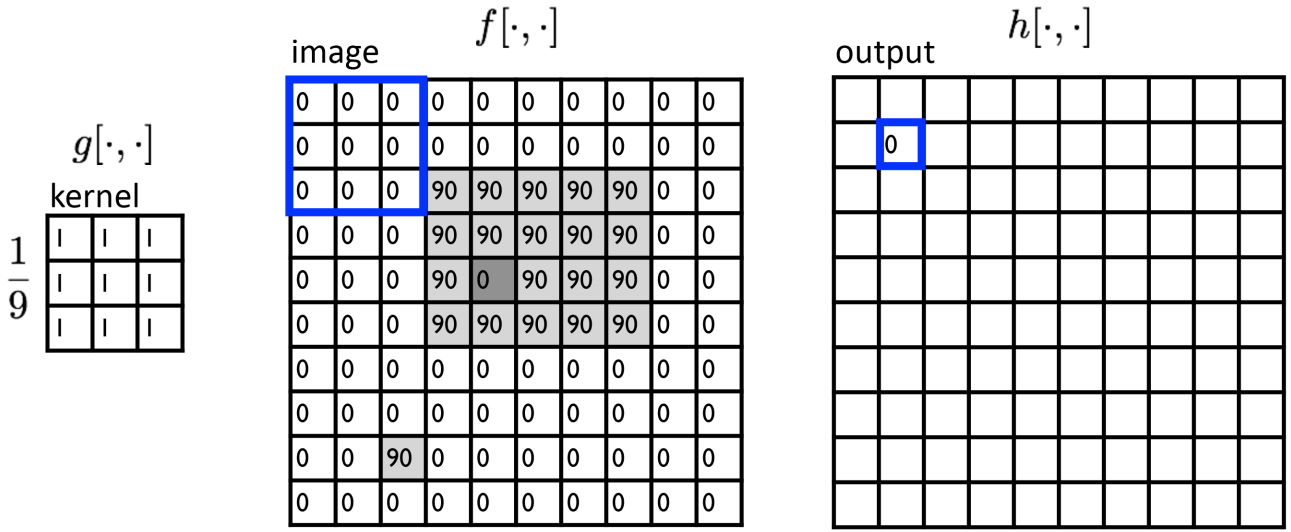
note that we assume that the kernel coordinates are centered

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output — filter — image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

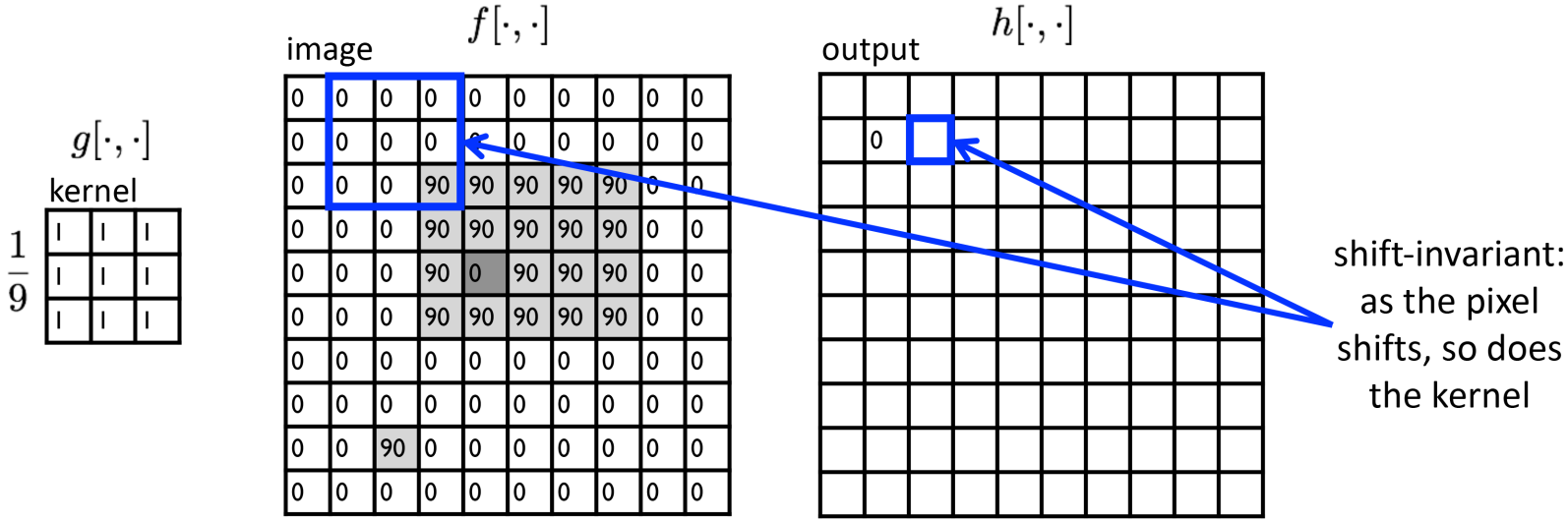


$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

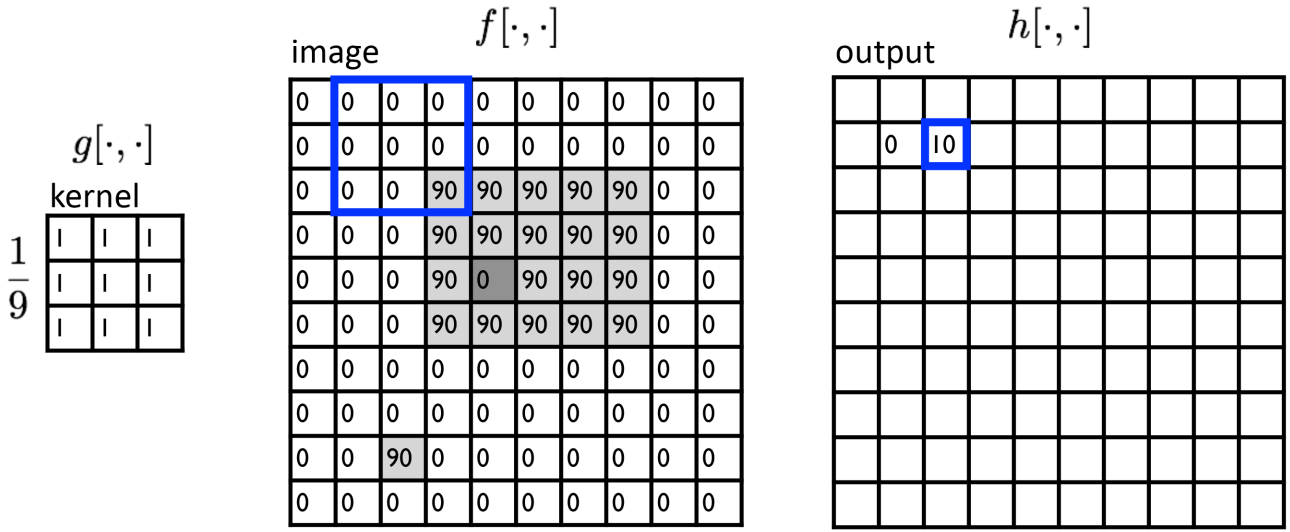


$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

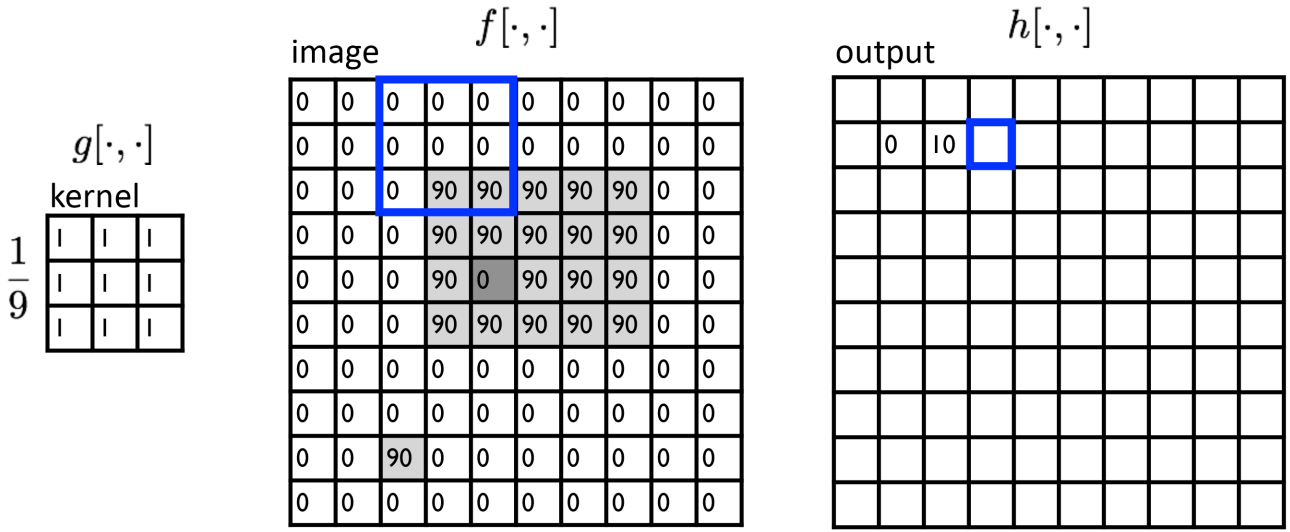


$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

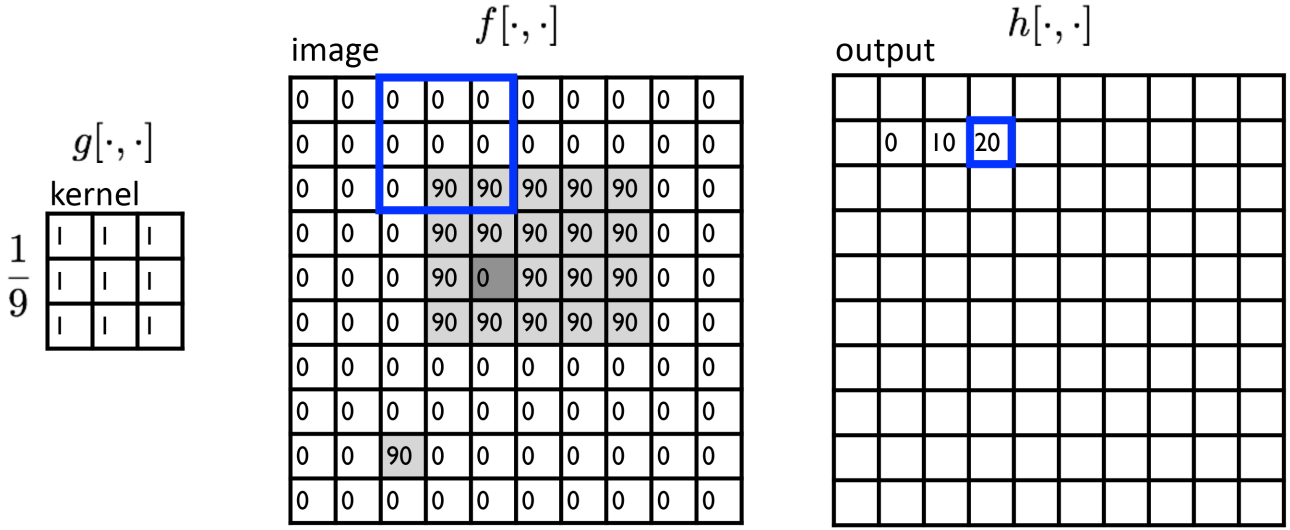


$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

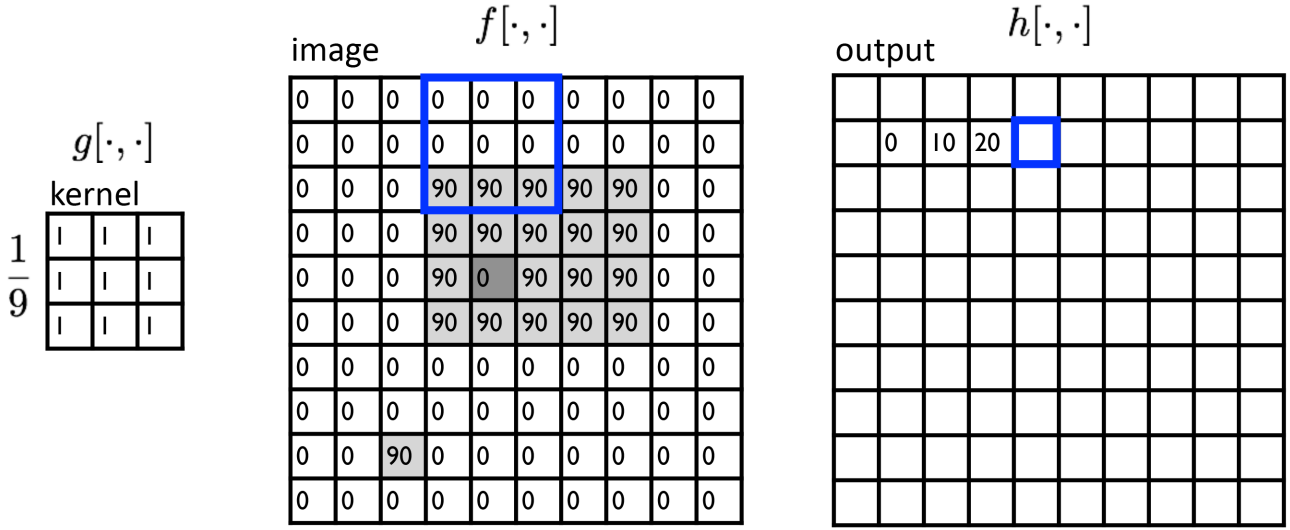


$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter



$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

$g[\cdot,\cdot]$

kernel

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[\cdot,\cdot]$

image

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot,\cdot]$

output

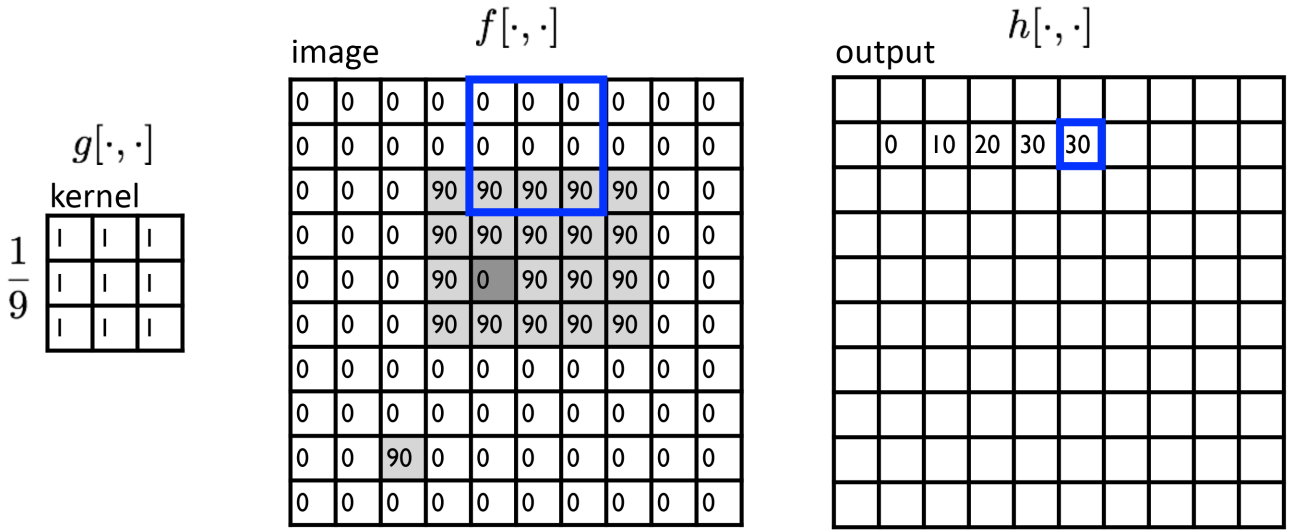
	0	10	20	30					

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

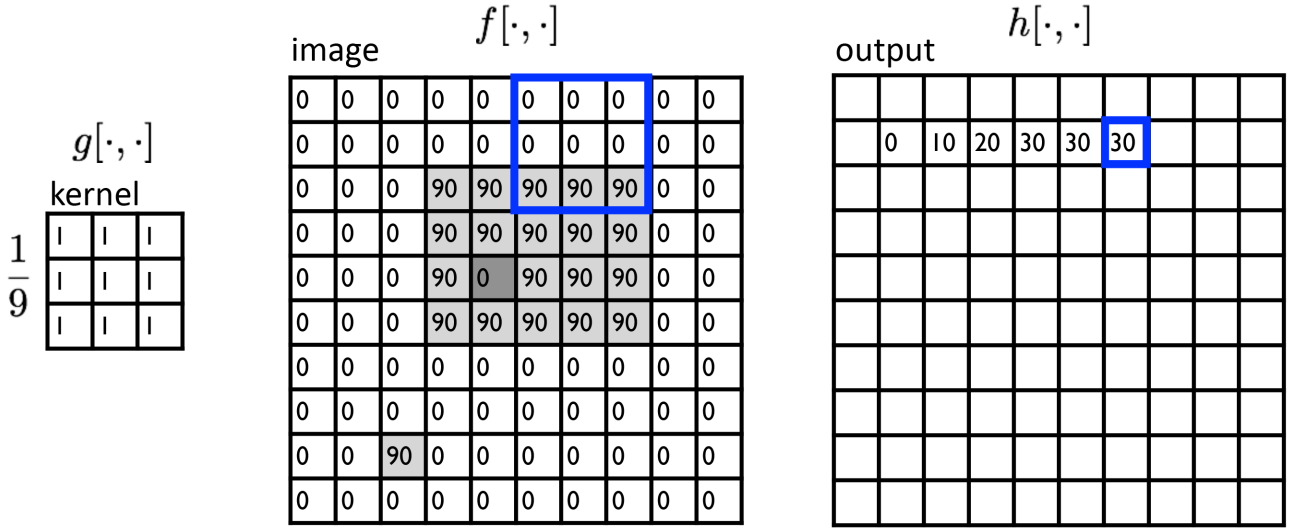


$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output — filter — image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter



$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output — filter — image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

$g[\cdot,\cdot]$

kernel

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[\cdot,\cdot]$

image

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot,\cdot]$

output

	0	10	20	30	30	30	20	10	
	0	20	40						

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

$g[\cdot,\cdot]$

kernel

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[\cdot,\cdot]$

image

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot,\cdot]$

output

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30							

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

$g[\cdot,\cdot]$

kernel

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[\cdot,\cdot]$

image

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot,\cdot]$

output

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	0	10	20	30	30	30	20	10	
	10	10	10	10	0	0	0	0	
	10								

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Let's run the box filter

$g[\cdot,\cdot]$

kernel

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[\cdot,\cdot]$

image

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot,\cdot]$

output

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	0	10	20	30	30	30	20	10	
	10	10	10	10	0	0	0	0	
	10	10	10	10	0	0	0	0	

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output — filter — image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

… and the result is

$g[\cdot,\cdot]$

kernel

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$f[\cdot,\cdot]$

image

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot,\cdot]$

output

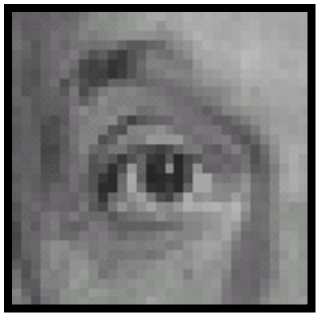
	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	0	10	20	30	30	30	20	10	
	10	10	10	10	0	0	0	0	
	10	10	10	10	0	0	0	0	

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k, n+l]$$

output filter image (signal)

Source -- http://16385.courses.cs.cmu.edu/fall2022/lecture/filtering/

Linear filters: examples

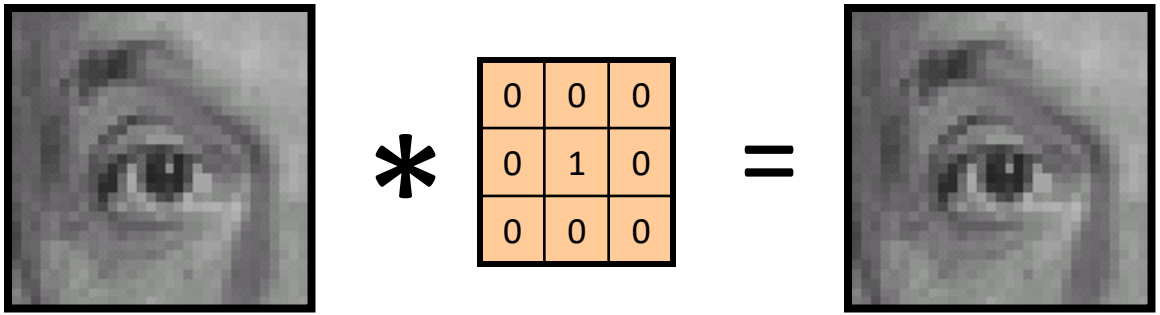


Original

$$*\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \; ?$$

[Slide credit: David Forsyth]

Linear filters: examples

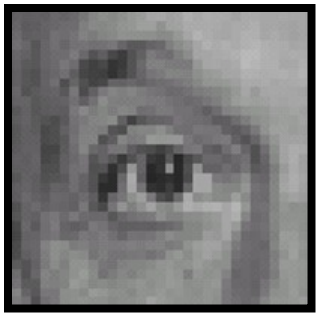


Original

Identical image
(no change)

[Slide credit: David Forsyth]

Linear filters: examples

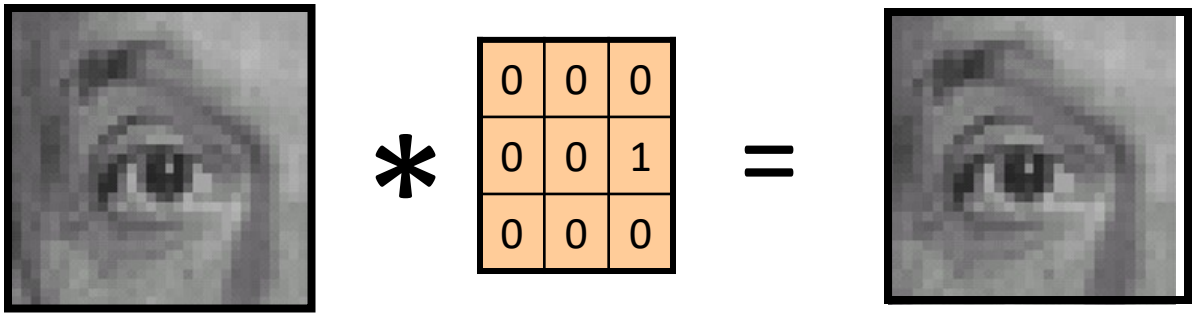


Original

$$*\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} = \; ?$$

[Slide credit: David Forsyth]

Linear filters: examples



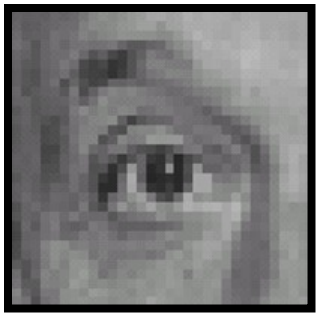
Original

Shifted left by 1 pixel

Note: mathematically speaking, this slide is doing cross-correlation and not convolution (for convenience, modern libraries do cross-correlation but call these convolution filters)

[Slide credit: David Forsyth]

Linear filters: examples

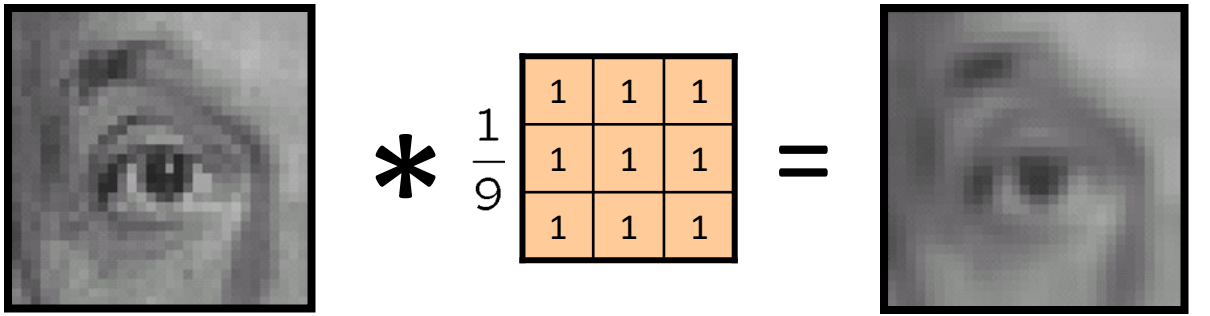


Original

$$* \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \; ?$$

[Slide credit: David Forsyth]

Linear filters: examples

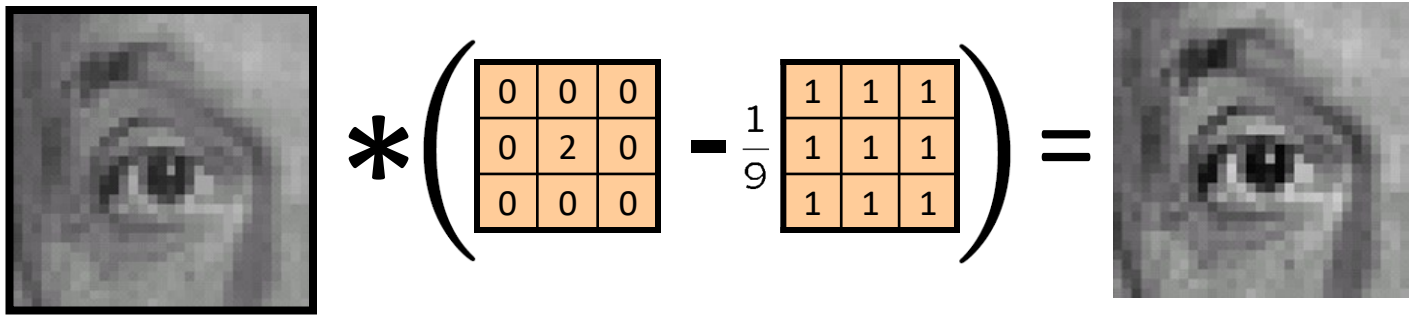


Original

Blur (with a box filter)

[Slide credit: David Forsyth]

Linear filters: examples



Original

Sharpening filter
(accentuates edges)

Edge filter

$$\frac{\partial \mathbf{I}}{\partial x} \simeq \mathbf{I}(x,y) - \mathbf{I}(x-1,y)$$

g[m,n]

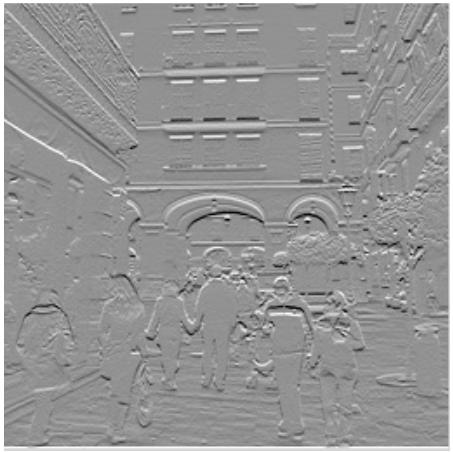


$\otimes$ [-1, 1] = h[m,n]

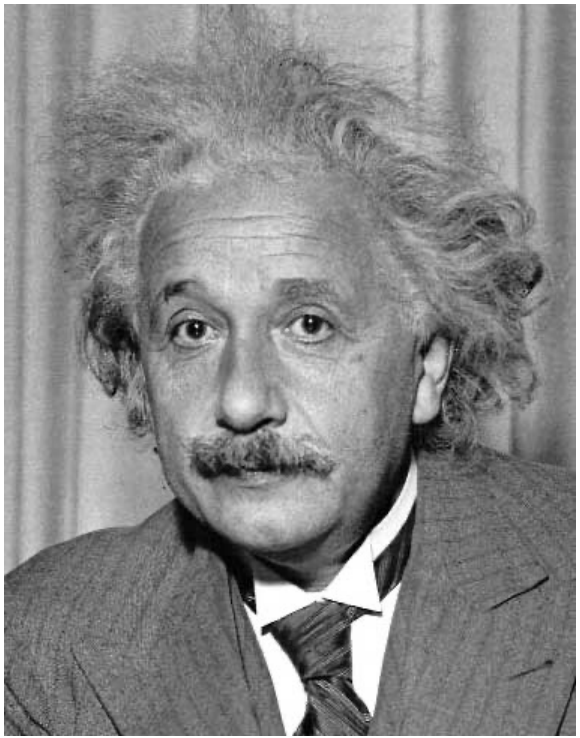


f[m,n]

$\otimes$ $[-1, 1]^T$ =

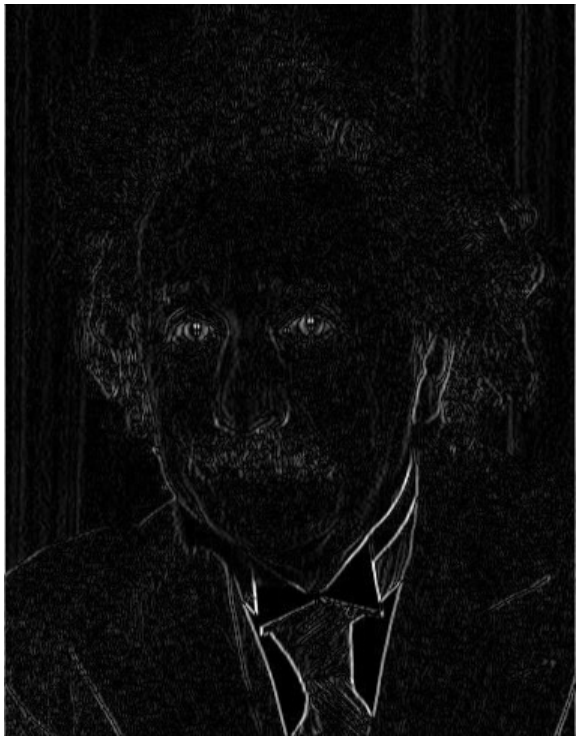


Derivative filters



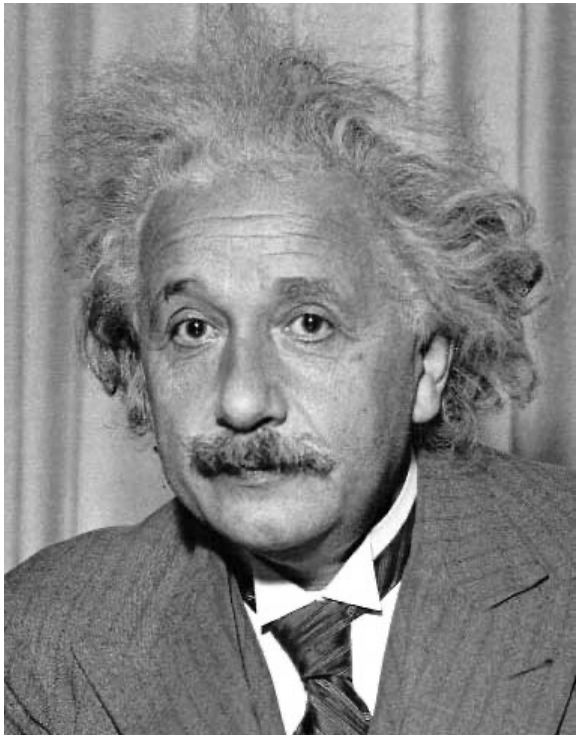
1	0	-1
2	0	-2
1	0	-1

Sobel



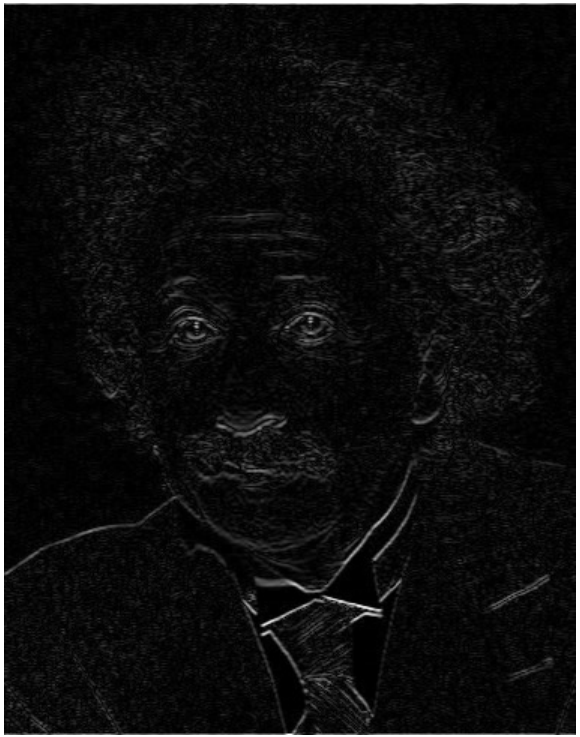
Vertical Edge
(absolute value)

Derivative filters



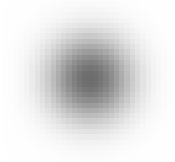
1	2	1
0	0	0
-1	-2	-1

Sobel



Horizontal Edge
(absolute value)

Motion blur filter



*



=



*



=



[Slide credit: Bastian Goldlücke]

Cross-Correlation vs. Convolution

Cross-Correlation

$$I'(x, y) = \sum_{j=-k}^{k} \sum_{i=-k}^{k} K(i, j) I(x + i, y + j)$$

Convolution

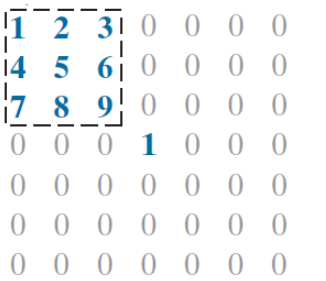
$$I'(x, y) = \sum_{j=-k}^{k} \sum_{i=-k}^{k} K(i, j) I(x - i, y - j)$$

$$= \sum_{j=-k}^{k} \sum_{i=-k}^{k} K(-i,-j) I(x + i, y + j)$$

means flipping the kernel vertically and horizontally

If kernel K is

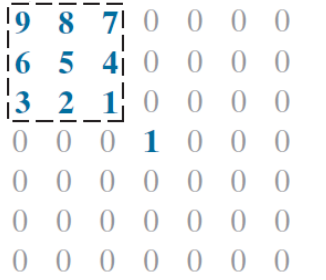
1	2	3
4	5	6
7	8	9



Correlation result



Convolution result



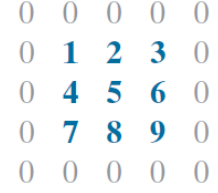


image source: https://towardsdatascience.com/convolution-vs-correlation-af868b6b4fb5/

if *K(i,j) = K(-i, -j)*, which means the kernel is symmetric, then Cross-Correlation is the same as Convolution

Mathematical definition of image convolution

- Image $f(x, y)$ and kernel $k(x, y)$
- Continuous form, $(x, y) \in \mathbb{R}^2$

$$k(x, y) * f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} k(u, v) f(x - u, y - v) du dv$$

- Discrete form, kernel size (W_k, H_k), Image size (W_f, H_f),

$$k(x, y) * f(x, y) = \sum_{i=0}^{W_k - 1} \sum_{j=0}^{H_k - 1} k(i, j) f(x - i, y - j)$$

- Commutative:

$$k(x, y) * f(x, y) = f(x, y) * k(x, y)$$

Mathematical definition of cross correlation

- Image $f(x, y)$ and kernel $k(x, y)$
- Continuous form, $(x, y) \in \mathbb{R}^2$

$$k(x,y) \otimes f(x,y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} k(u,v) f(x+u, y+v) du dv$$

- Discrete form, kernel size (W_k, H_k), Image size (W_f, H_f),

$$k(x,y) \otimes f(x,y) = \sum_{i=0}^{W_k - 1} \sum_{j=0}^{H_k - 1} k(i,j) f(x+i, y+j)$$

- If $k = f$, the operation is called auto-correlation
- Relation with convolution

$$k(x,y) \otimes f(x,y) = k(-x,-y) * f(x,y)$$

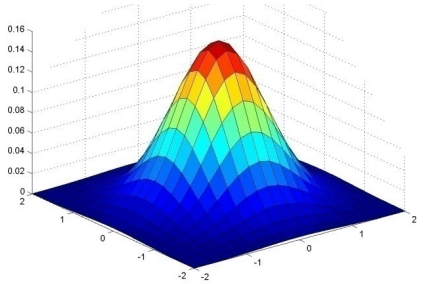
Image filtering

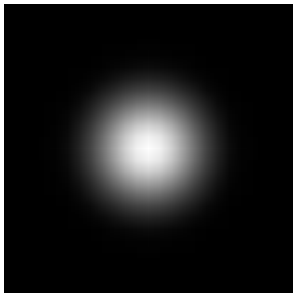
We can use image filtering for:

- e.g., Enhance images
  - De-noise, resize, increase contrast, etc.
- e.g., Extract information from images
  - Texture, edges, distinctive points, etc.
- e.g., Detect patterns
  - Template matching

Gaussian filter

- Weight contributions of neighboring pixels by nearness





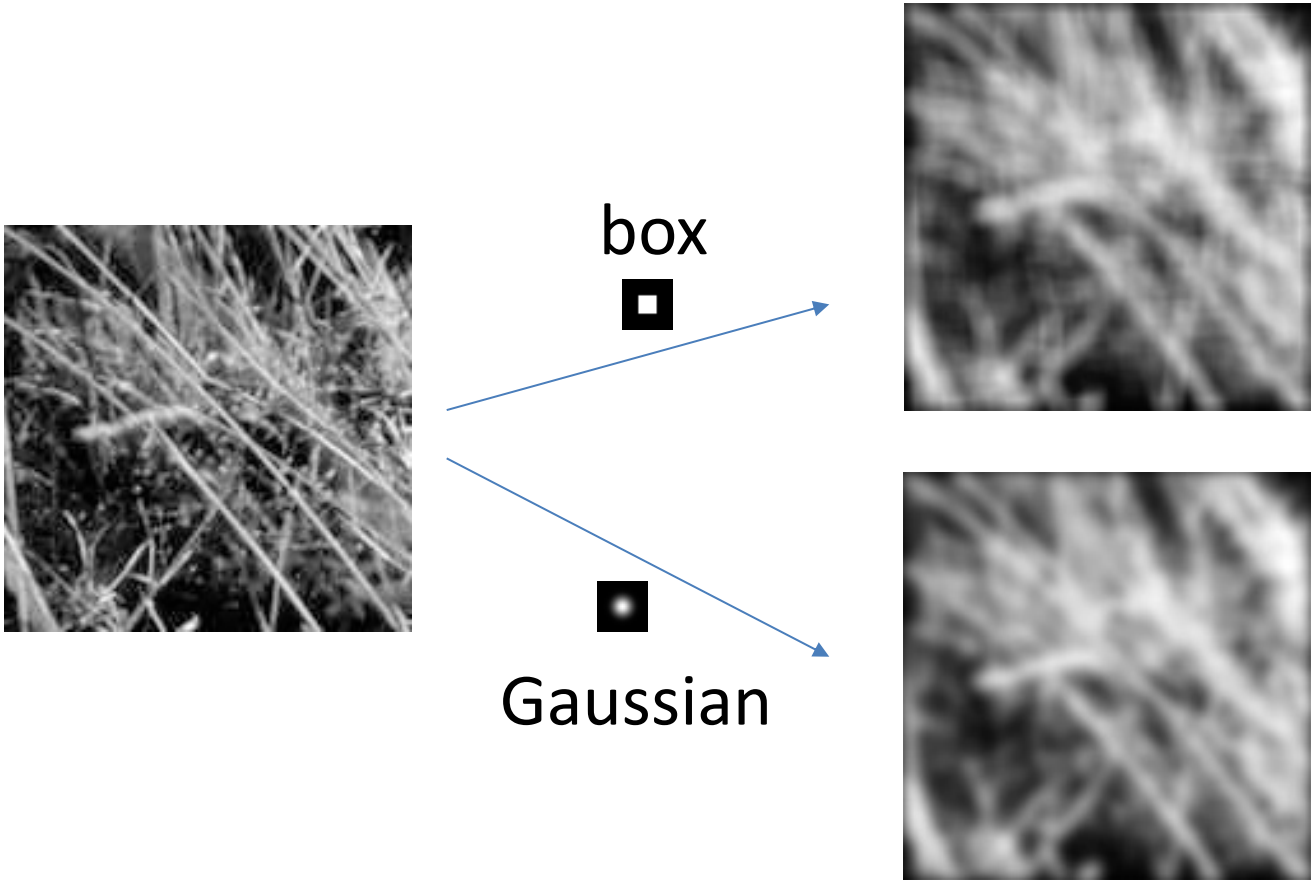
0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$

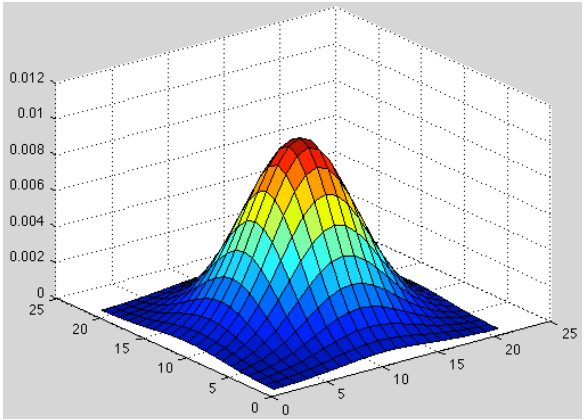
$$G(x, y; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Slide credit: Christopher Rasmussen

Smoothing with box or Gaussian filter

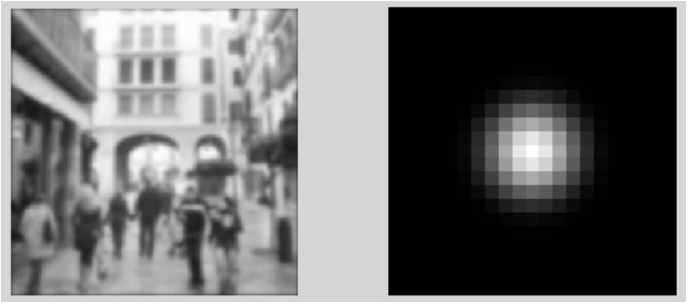


Gaussian filter: change scales

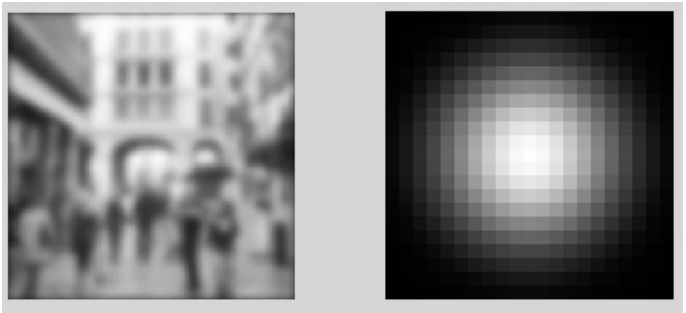




σ=1



σ=2



σ=4

Properties of Gaussian filter

- Rotational symmetry treats features of all orientations equally (isotropy).

- Convolving a Gaussian filter (mean 0, variance σ_1^2) with another Gaussian filter (mean 0, variance σ_2^2) gives also a Gaussian filter (mean 0, variance $\sigma_1^2 + \sigma_2^2$).

$$G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2+\sigma_2^2}}$$

Separability of the Gaussian filter

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2}\exp^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$= \left(\frac{1}{\sqrt{2\pi}\sigma}\exp^{-\frac{x^2}{2\sigma^2}}\right)\left(\frac{1}{\sqrt{2\pi}\sigma}\exp^{-\frac{y^2}{2\sigma^2}}\right)$$

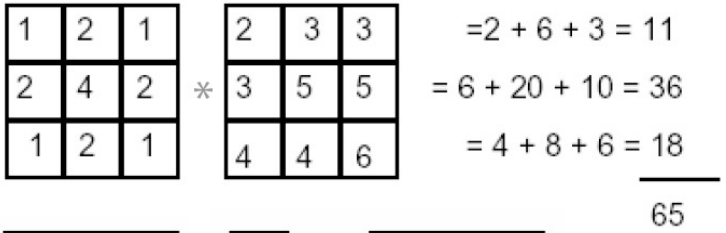
The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

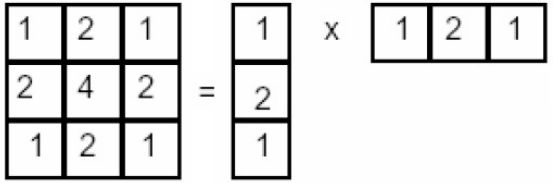
Source: D. Lowe

Separability example

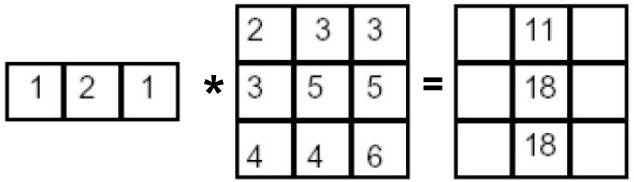
2D convolution
(center location only)



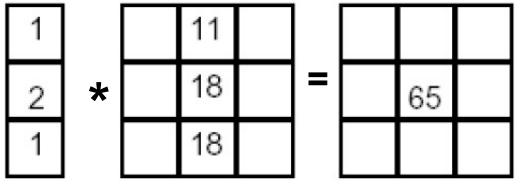
The filter factors
into a product of 1D
filters:



Perform convolution
along rows:



Followed by convolution
along the remaining column:



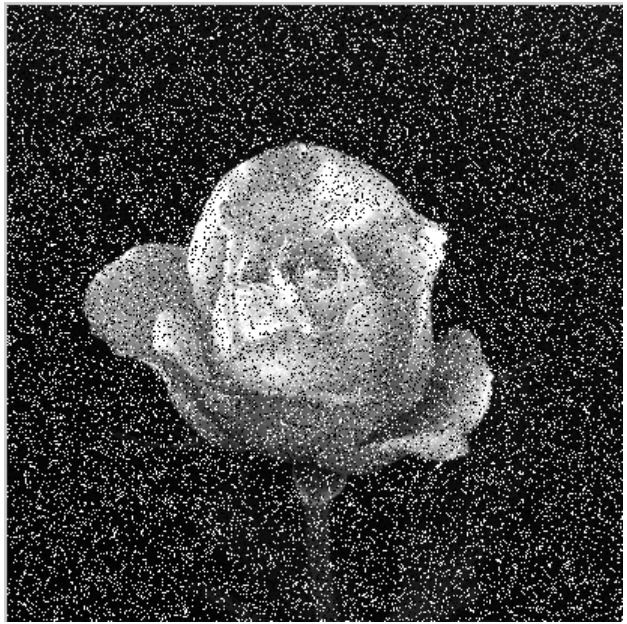
Source: K. Grauman

Outline

- Neighborhood image processing
  - Connected Component Analysis
  - Mathematical Morphology (Dilation, Erosion, Opening, Closing)
- Linear Image Filtering
  - Cross-Correlation and Convolution
  - Frequently used Filters
  - Gaussian Filter
- **Nonlinear Image Filtering**
  - Median Filter
- Frequency Domain Analysis
  - Fourier Transform

Reduce Salt-and-Pepper Noise

- Noise with random occurrence of white and black pixels (signal independent).
  Can be due to transmission errors, corrupted pixels, faulted memory locations, etc.



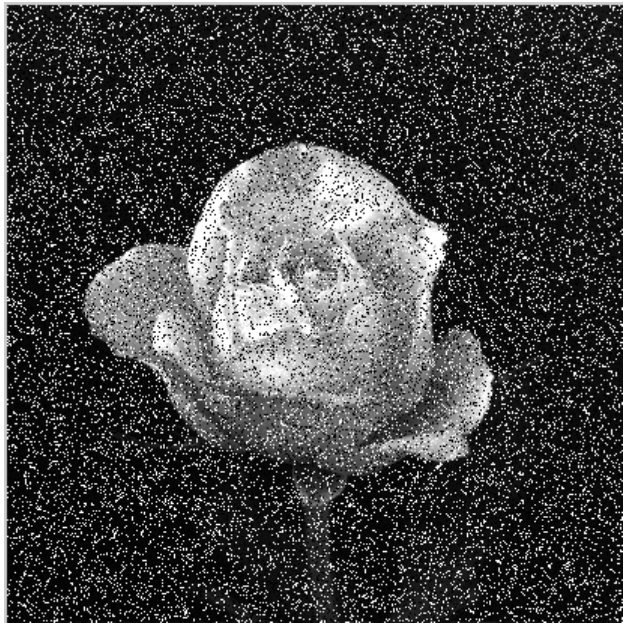
Use Median Instead of Average

- Idea: we can use median to get rid of outliers

1, 2, 5, 7, 80

Mean = 19

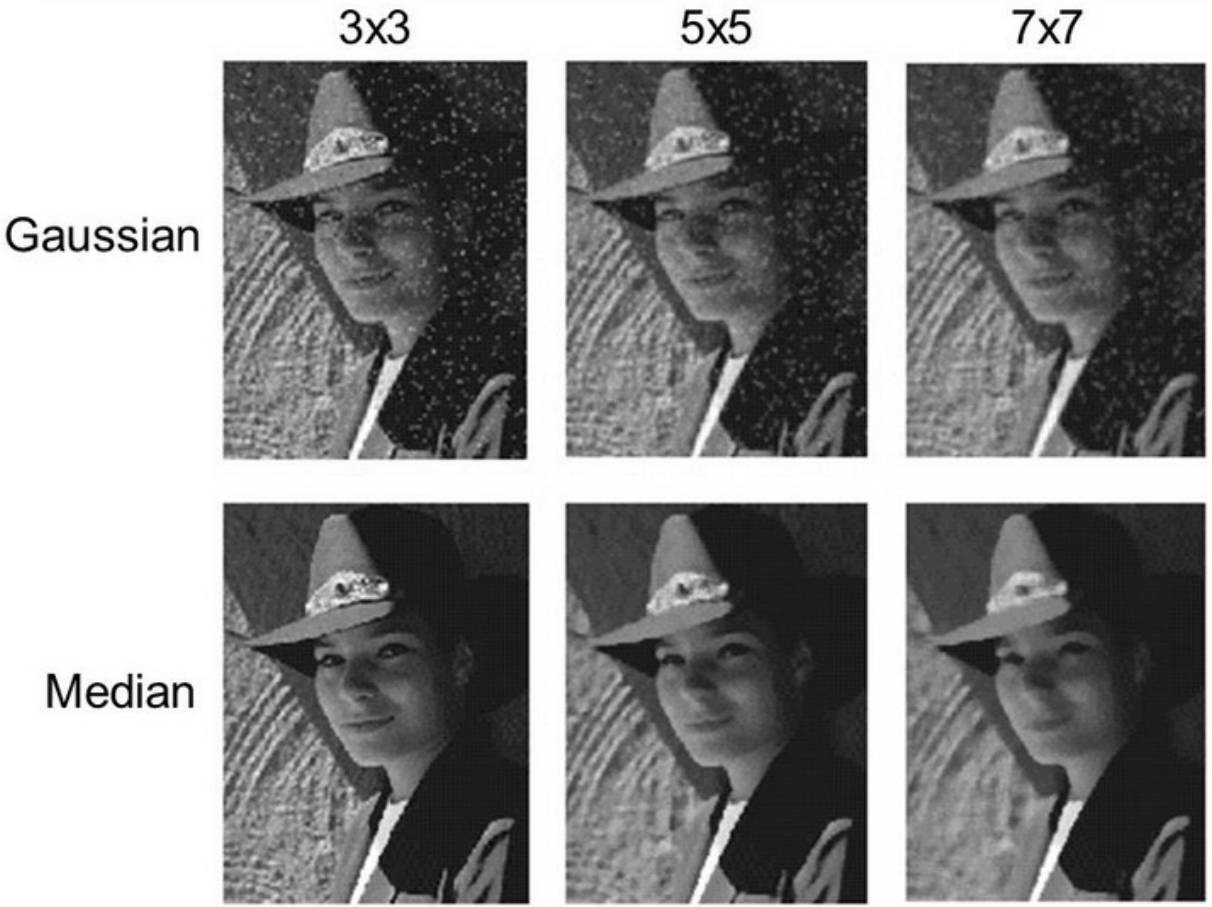
Median = 5



median filter



Gaussian vs. Median Filtering



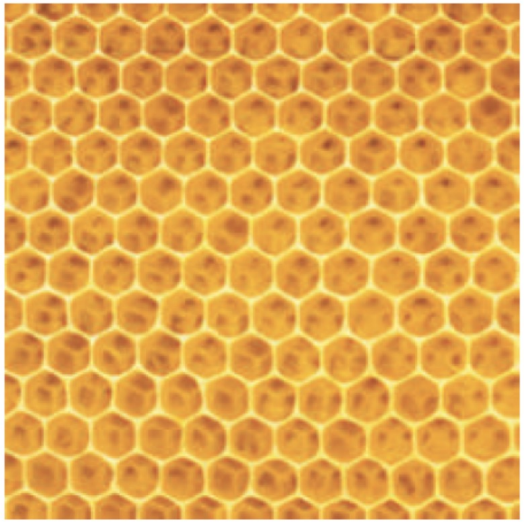
Outline

- **Neighborhood image processing**
  - Connected Component Analysis
  - Mathematical Morphology (Dilation, Erosion, Opening, Closing)
- **Linear Image Filtering**
  - Cross-Correlation and Convolution
  - Frequently used Filters
  - Gaussian Filter
- **Nonlinear Image Filtering**
  - Median Filter
- **Frequency Domain Analysis**
  - Fourier Transform

The frequency perspective

We can look at images from the frequency perspective.

The visual world contains:



Repetitive structures



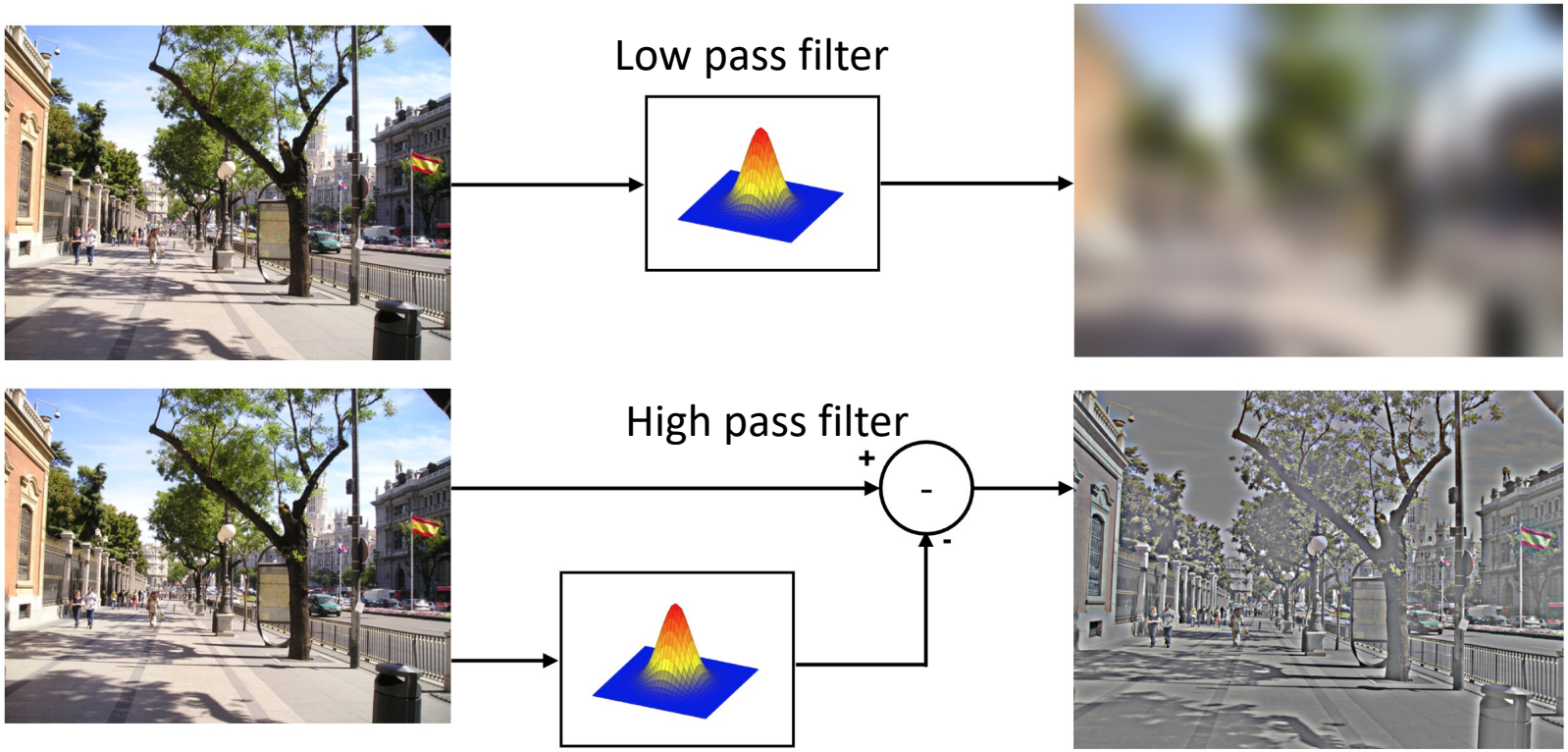
…that repeat quickly



…or slowly

Source -- https://www.cs.cornell.edu/courses/cs5670/2025fa/slides/lec4-frequency.pdf

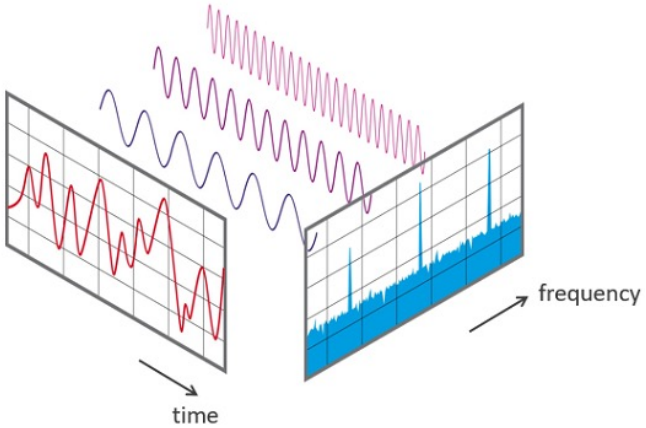
The frequency perspective

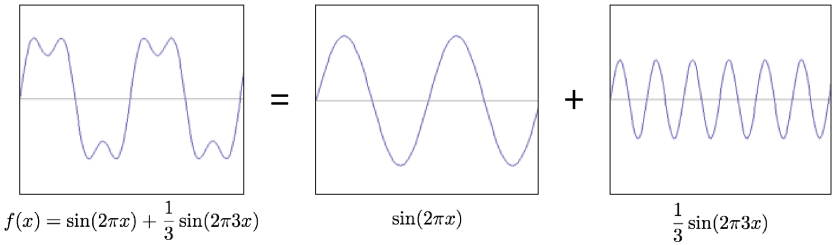


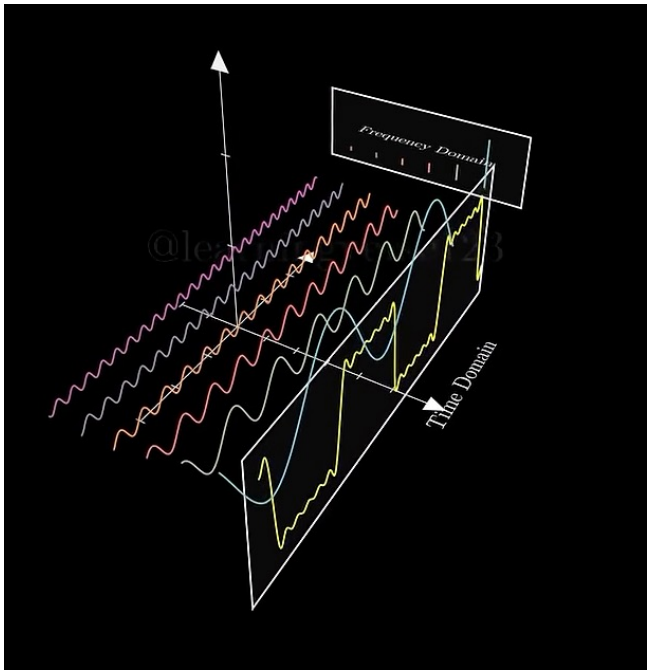
Source -- https://www.cs.cornell.edu/courses/cs5670/2025fa/slides/lec4-frequency.pdf

Fourier Analysis

We can decompose the signal into a combination of sinusoids (i.e., sine waves with various phases, amplitudes, and frequencies).





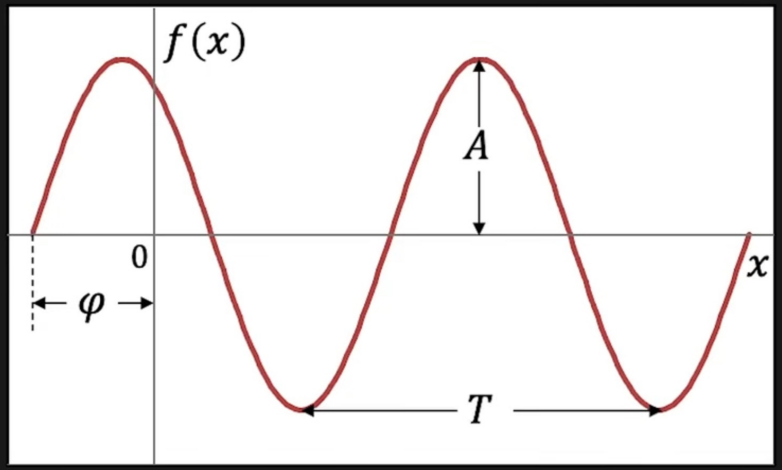


[learningverse123 - YouTube]

Source -- https://www.cs.cmu.edu/~16385/lectures/lecture3.pdf

Basics of sinusoids

$$f(x) = A\sin(2\pi ux + \varphi)$$



A: Amplitude

T: Period

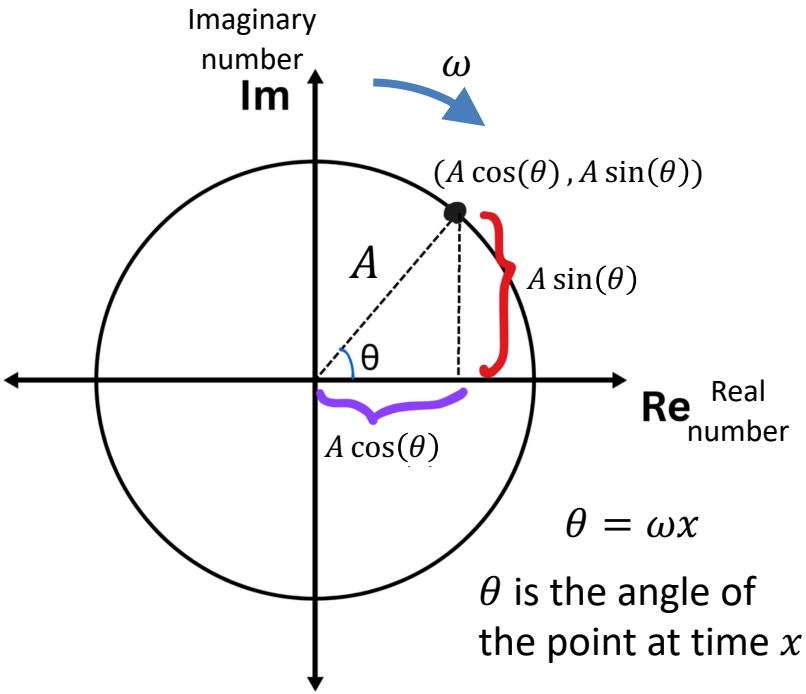
φ: Phase

u: Frequency $(1/T)$

Source -- https://www.youtube.com/watch?v=tEzgtbnbXgQ

Basics of polar coordinate

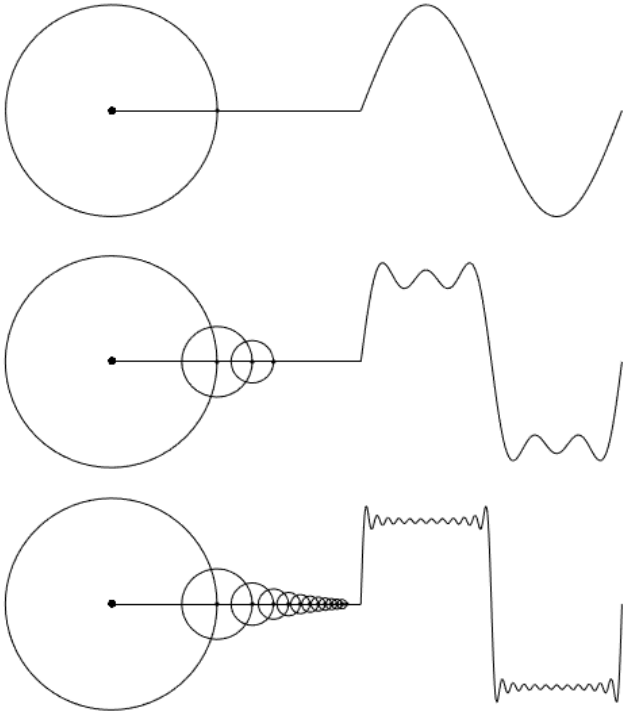
Think about a rotating point on a complex plane with radius A (amplitude), time x, and angular speed ω.



$\theta = \omega x$

θ is the angle of the point at time x

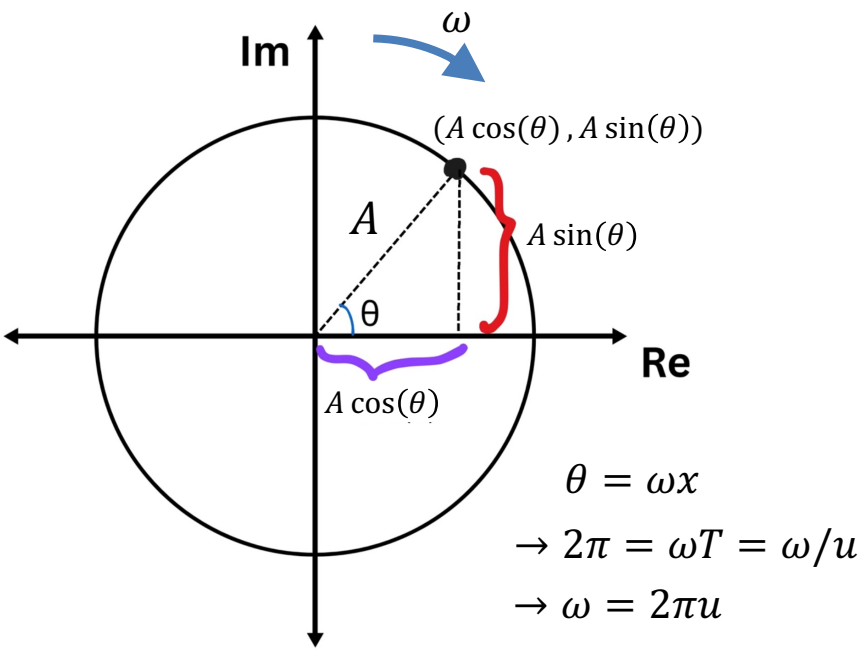
Source -- https://www.youtube.com/watch?v=PcYFnVBS_bg



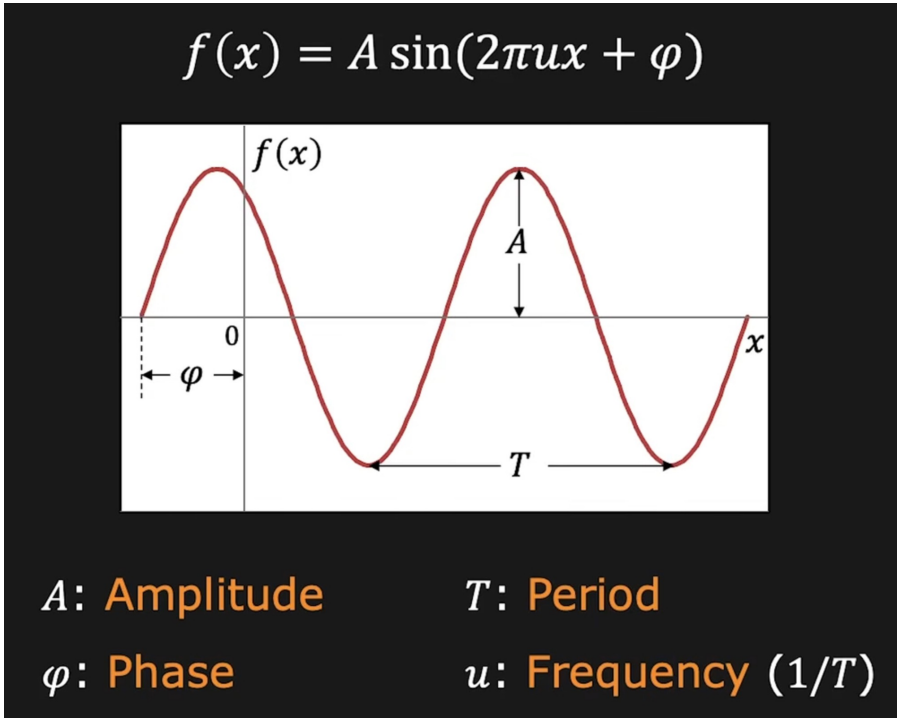
Source -- https://www.designcoding.net/fourier-series/

Connecting sinusoids and polar coordinate

Think about a rotating point on a complex plane with radius A (amplitude), time x, and angular speed ω, which starts rotating at angle ϕ (phase) on the circle.



$$\theta = \omega x$$
$$\rightarrow 2\pi = \omega T = \omega / u$$
$$\rightarrow \omega = 2\pi u$$



Source -- https://www.youtube.com/watch?v=PcYFnVBS_bg

Source -- https://www.youtube.com/watch?v=tEzgtbnbXgQ

Fourier Transform

Spatial domain x → Frequency domain u : Fourier Transform

$$F(u) = \int_{-\infty}^{\infty} f(x) e^{-i2\pi ux} dx$$

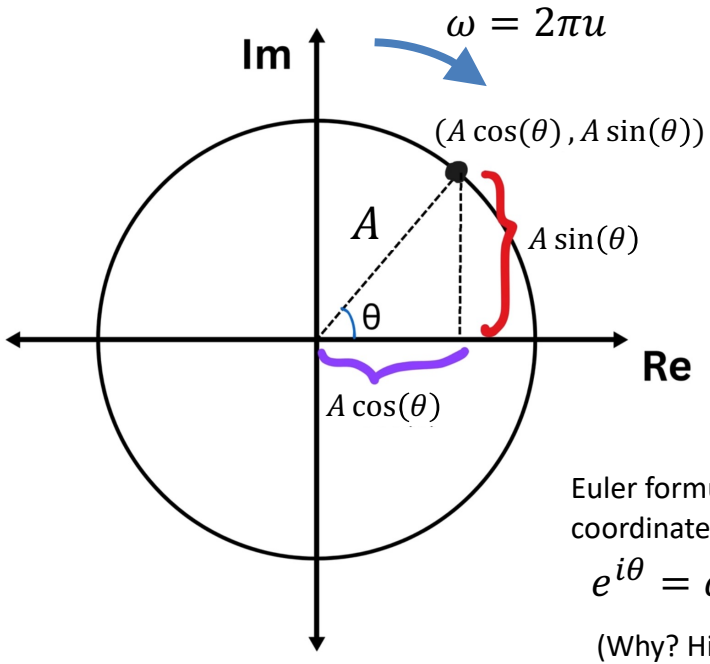
x ← u : Inverse Fourier Transform

$$f(x) = \int_{-\infty}^{\infty} F(u) e^{i2\pi ux} du$$

$$i = \sqrt{-1}$$

This is the continuous case for 1D

Where is the connection to the "summation of sinusoids" idea?



$\theta = 2\pi ux$
(see the previous slide for derivation)

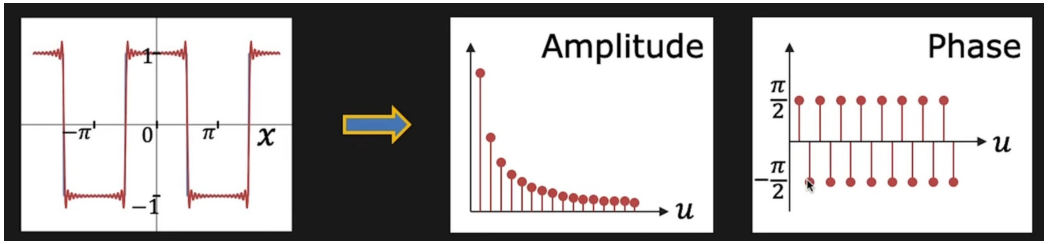
Euler formula on the polar coordinate of the complex plane

$$e^{i\theta} = cos\theta + i\, sin\theta$$

(Why? Hint: Try Taylor series expansion of these terms)

Fourier Transform

We can get amplitude and phase from the Fourier transformed frequency domain.

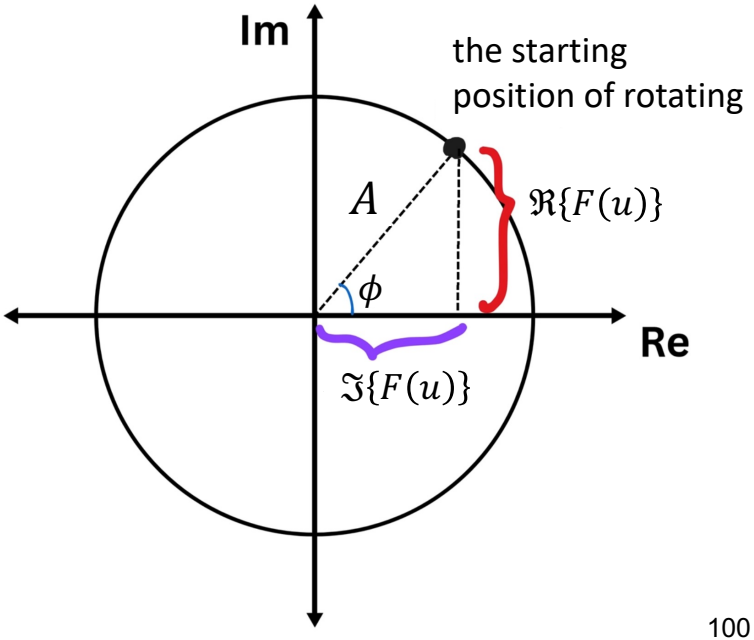


$$F(u) = \int_{-\infty}^{\infty} f(x) e^{-i2\pi ux} dx$$

$$F(u) = \Re\{F(u)\} + i \cdot \Im\{F(u)\}$$

Amplitude: $A(u) = \sqrt{\Re\{F(u)\}^2 + \Im\{F(u)\}^2}$

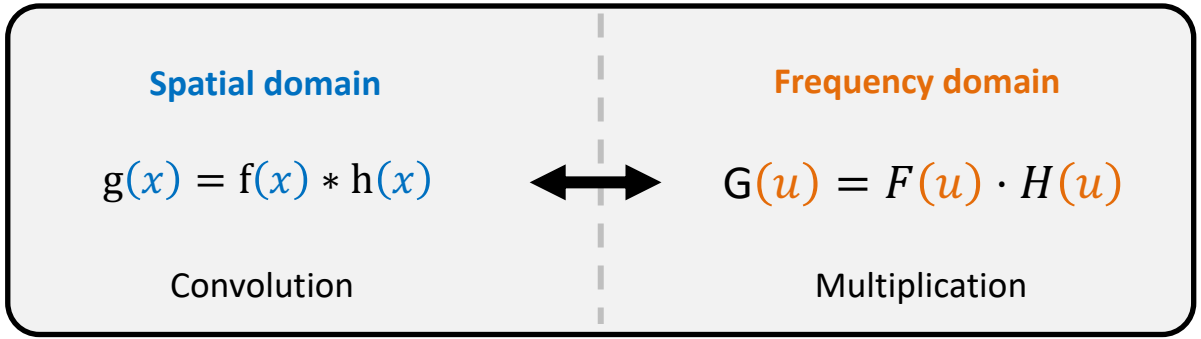
Phase: $\phi(u) = atan2(\Re\{F(u)\}, \Im\{F(u)\})$



Convolution Theorem

The convolution theorem

1. The Fourier transform of the convolution of two functions is the product of their Fourier transforms
2. The Fourier transform of the product of two functions is the convolution of the Fourier transforms



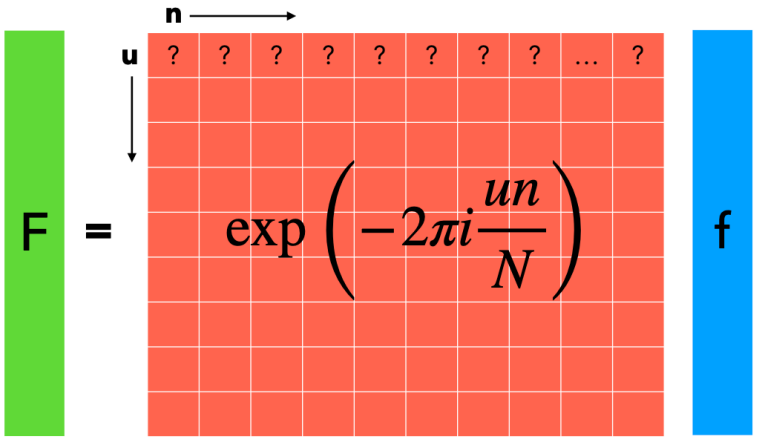
Why? See https://www.youtube.com/watch?v=kQUfKf-vPdI for the proof

The Discrete Fourier Transform (1D)

Discrete Fourier Transform (DFT) transforms a signal *f[n]* into *F[u]* as:

$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi i \frac{un}{N}\right)$$

Discrete Fourier Transform (DFT) is a linear operator. In matrix form:



U_F : NxN matrix

source -- https://www.cs.cornell.edu/courses/cs5670/2025fa/slides/lec4-frequency.pdf

The Discrete Fourier Transform (2D)

1D Discrete Fourier Transform (DFT) transforms a signal $f[n]$ into $F[u]$ as:

$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi i \frac{un}{N}\right)$$

2D Discrete Fourier Transform (DFT) transforms an image $f[n,m]$ into $F[u,v]$ as:

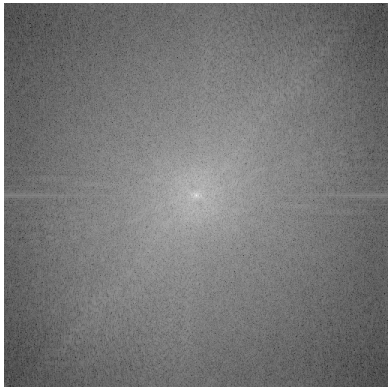
$$F[u,v] = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} f[n,m] \exp\left(-2\pi i \left(\frac{un}{N} + \frac{vm}{M}\right)\right)$$

source -- https://www.cs.cornell.edu/courses/cs5670/2025fa/slides/lec4-frequency.pdf

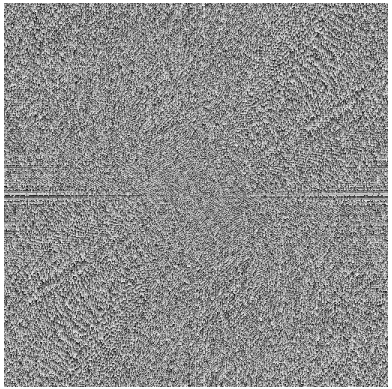
Fourier Space Visualization



Input

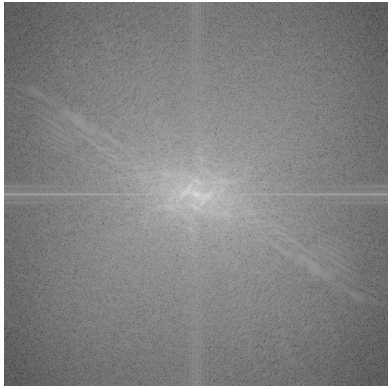


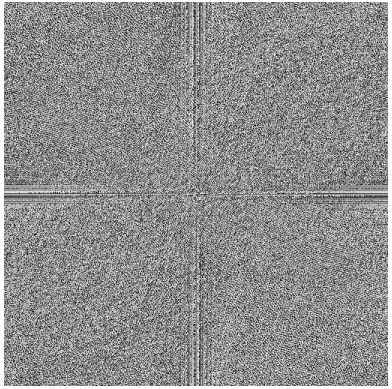
Magnitude/Amplitude



Phase



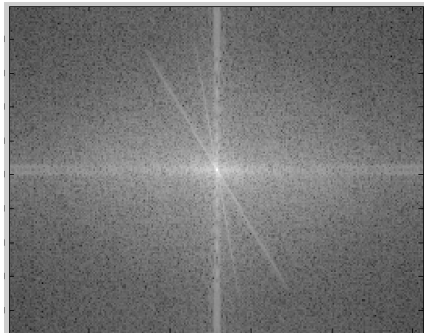




Source -- https://www.cs.cmu.edu/~16385/lectures/lecture3.pdf

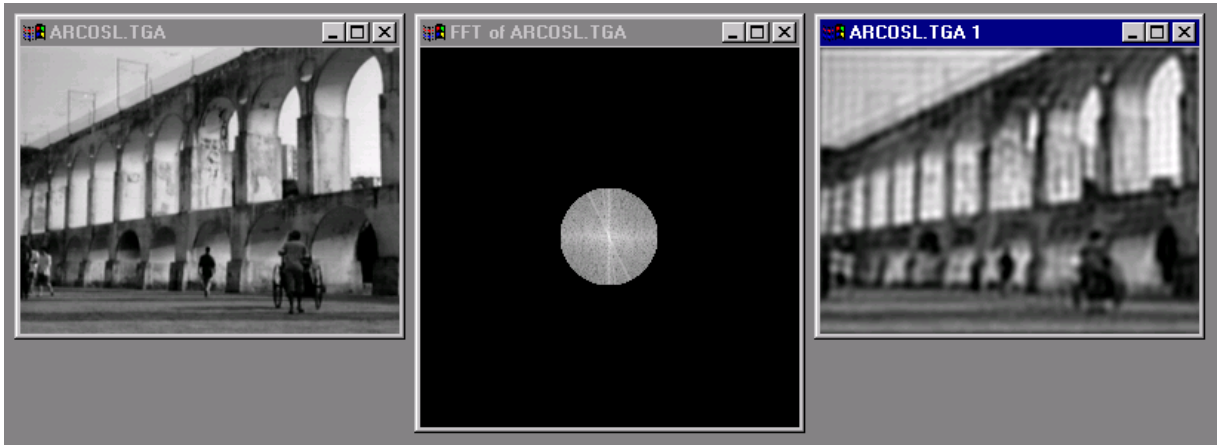
Low and High Pass filtering

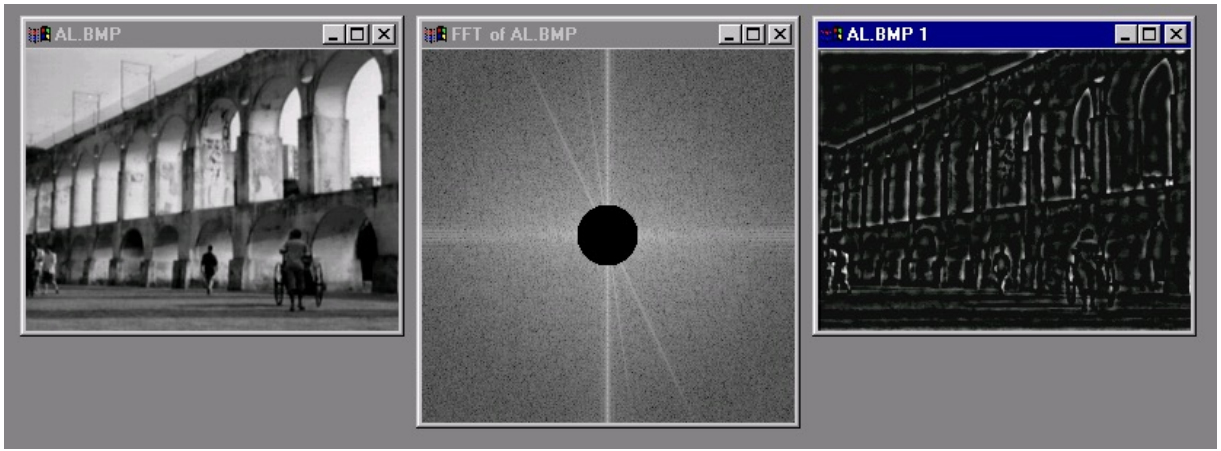




low pass

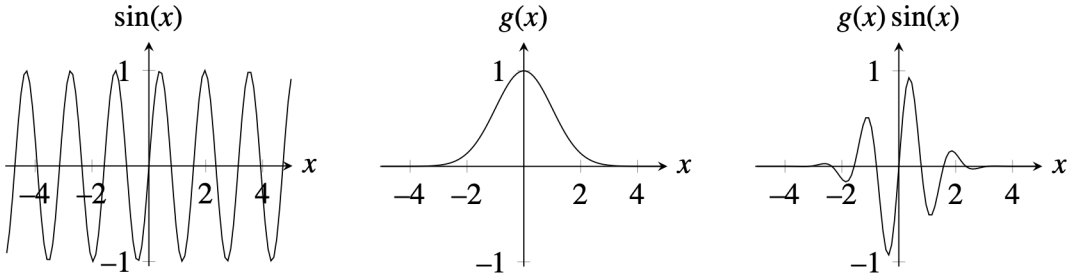
high pass



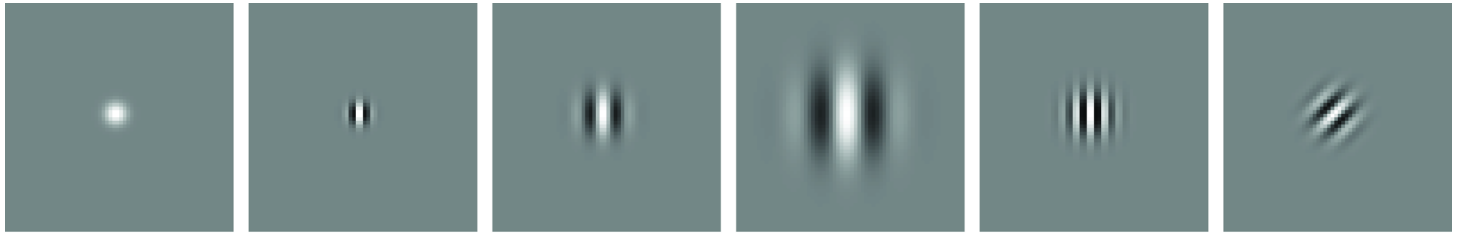


Gabor Filters

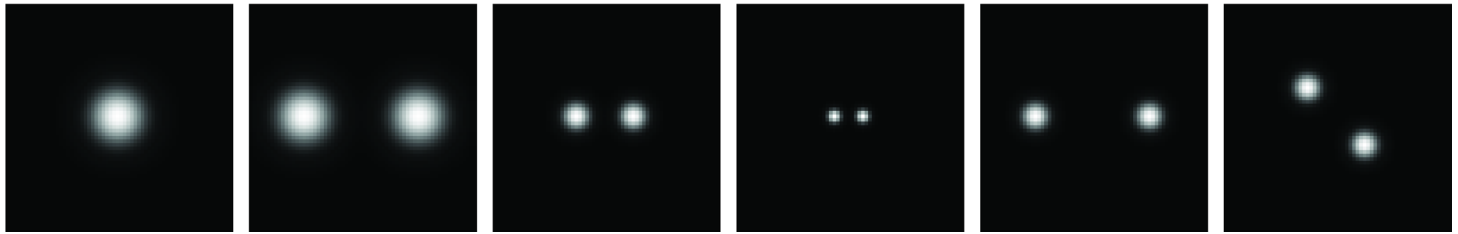
Gabor filter is the product of a Gaussian kernel and a sinusoid function, which is usually used as a filter bank for extracting features.



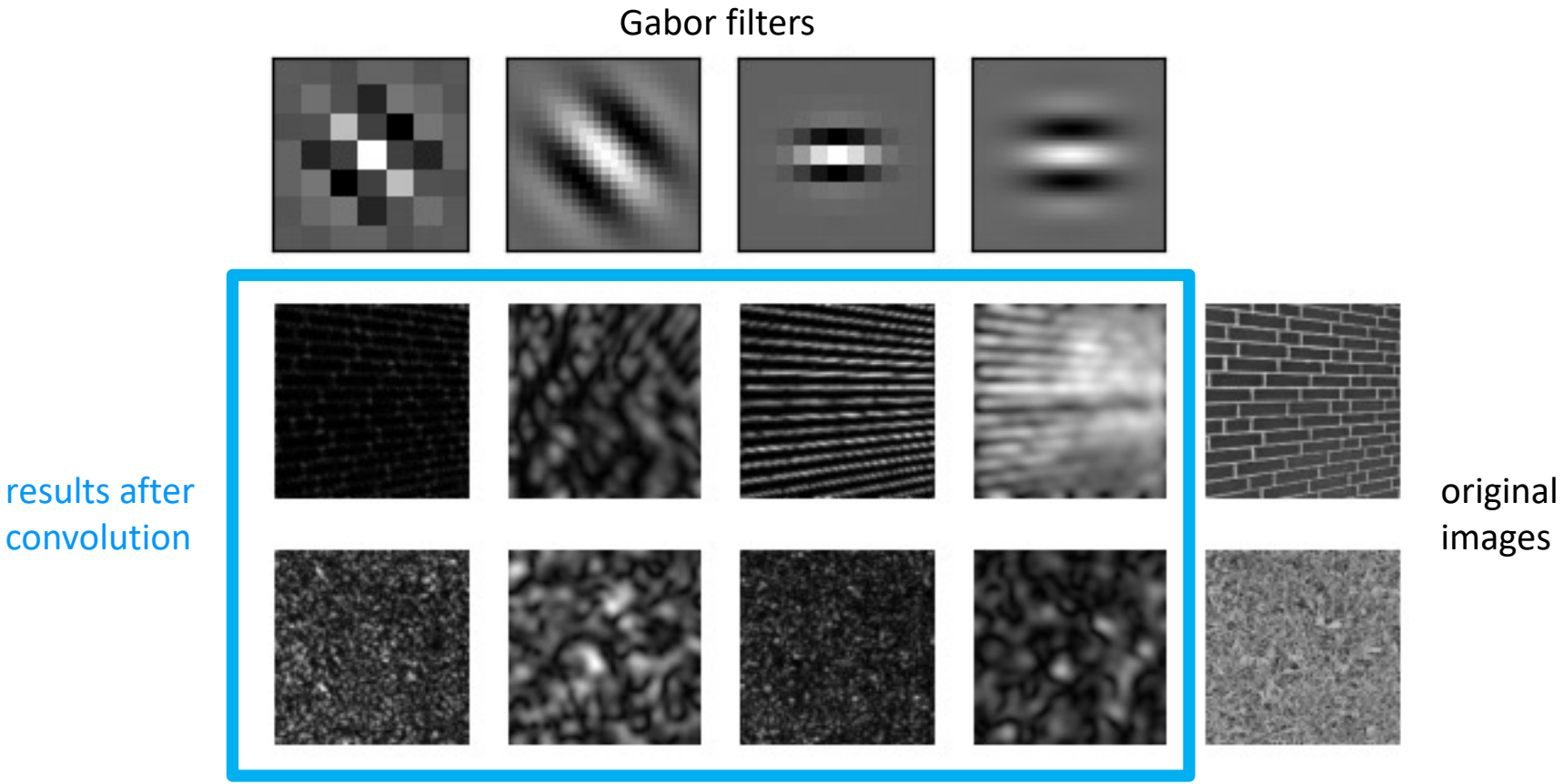
Gabor filters



Magnitudes after Fourier transforms

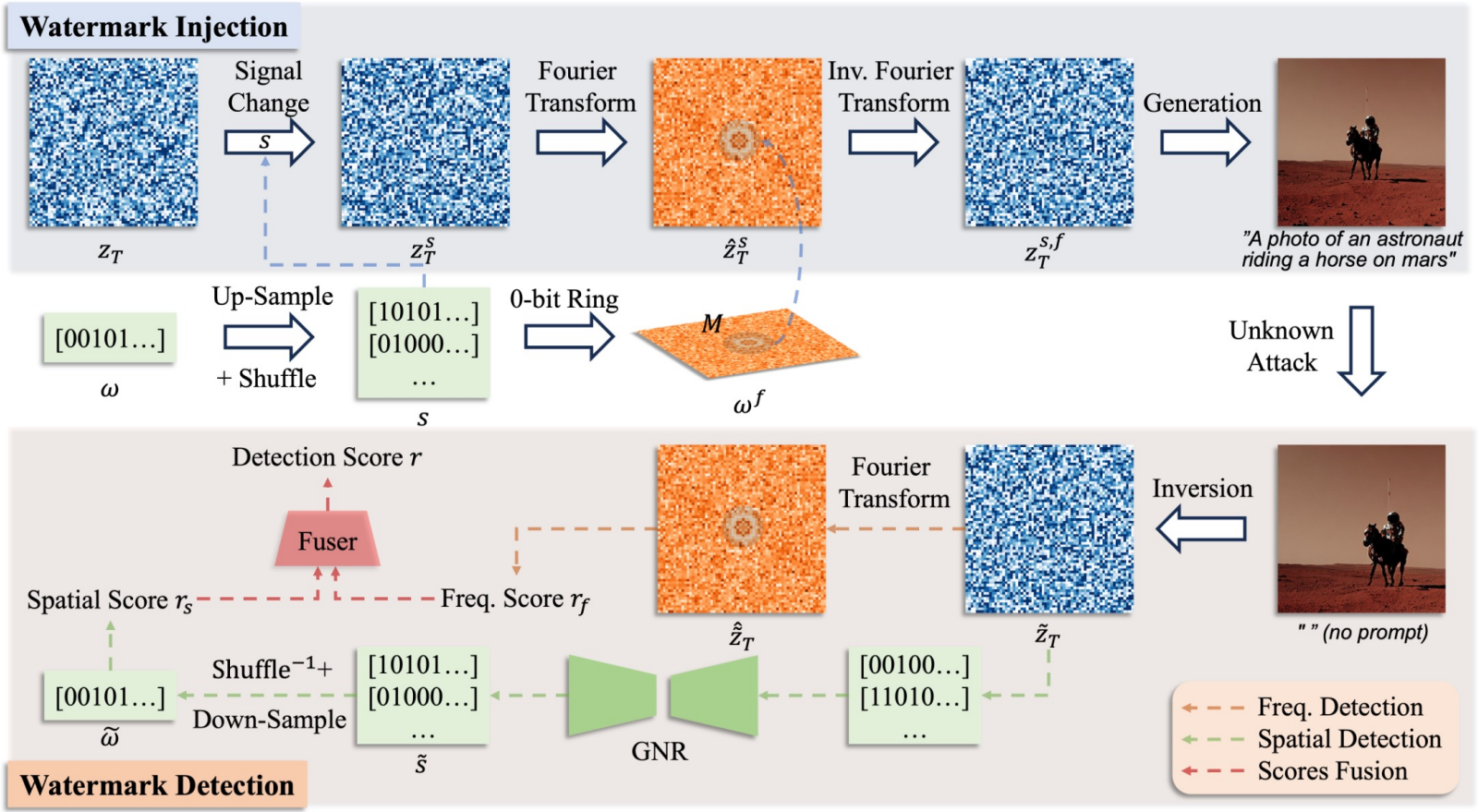


Gabor Filters



Source -- https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_gabor.html

Application: image watermarking and detection



source -- https://arxiv.org/pdf/2506.11444

Additional Resources

- Convolution
  - 3Blue1Brown: YouTube

- Fourier Transform
  - Fourier Transform Best Explanation (for Beginners)
  - Fourier Transform | Image Processing II
  - Image Filtering in Frequency Domain | Image Processing II
  - Convolution Theorem | Image Processing II

Disclaimer

Many of the slides used here are obtained from online resources (including many open lecture materials) without appropriate acknowledgement. They are used here for the sole purpose of classroom teaching. All the credit and all the copyrights belong to the original authors. You should not copy it, redistribute it, put it online, or use it for any other purposes than for this course.